

Cloud Computing Technologies

simi

February 2024

The course focuses on 4 main areas:

- Introduction to modern distributed systems
- Cloud computing Fundamentals
- Microservice Architecture Fundamentals
- Non-functional aspects of modern distributed systems

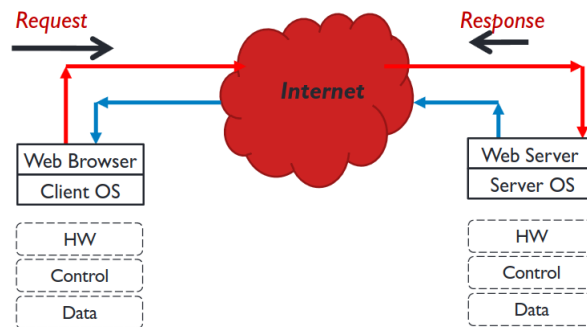
Distributed system paradigm:

- The basic trivia
- Definitions
- Properties
- Architectures
- Process
- Communication
- Consistency and replication

Let's start with some notion of distributed system, it's important to know some definitions that will be useful later in the course, then we'll see three elements starting from process, how a process could be distributed, somehow we have to separate the parallel to sequence (sequence process could be distributed too), then we'll see Properties and Architectures of distributed system. We always have to make reachable the process, making several servers and copies of the same services. Do we have the exactly the same service in US and Europe? maybe not for the different legislation, updates must be separated. Then we'll discuss some advanced architectures.

0.1 Basic

A very basic distributed system is composed by 2 computer talking each other, the example in the picture is a web connection. We have client and server, both are doing computational. We have in one side the operating system of the server and in the other side the operating system of the client, they shouldn't be the same, they just need to have some services on top that exchange data.



Computer systems executes:

- Stand-alone application
- Network application

and requires:

- HW: processor
- Control: software
- Data: information

Nowdays we have a lot of computational function, that are function that do computation without storing anything.

An intuitive definition of distributed system is a system where HW and/or Control and/or Data are distributed. This tells that it can be different level of distribution (just HW, just Control ...)

We also note that distribution is a three dimensional space, we can move in this three dimensional space.

0.2 HW (distributed processing system)

HW is basically the processing capability, the only HW we considerate are CPU and GPU. In distributed system there are always two or more computer, processing element is much more correct because as we said we only consider CPU and GPU. Each processing element have here own local memory and processor (even the local cache are considered memory). The computers have to be connected via networking communication.

So they have to process something to provide us something, like computer browsing. Processing and elaborated data separately for the same goal.

0.3 Data (distributed database)

We usually have distributed database, we have several replicas that are synchronized but data are spread. If client modify something in the data layer the server see the changes.

Data to be processed is itself distributed:

- Replication: multiple copies, we have a lot of problems, synchronization could be a very complex requirements for distributed system (also geographical distance)
- Partitioning: we usually have data stored in different location, for example in web browsing the client has the cache (cookies).

0.4 Control

Lets start with centralized architecture, we have HW, Control and Data on the same machine. In the orther side we have de Distributed Architecture in which everything is distributed, and in the middle we have Client-Side architecture in which mainly there are Distributed Control.



0.5 Centralizaion vs Distribution

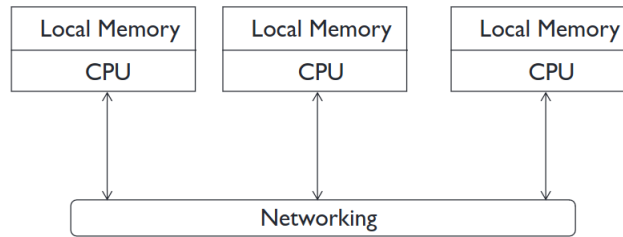
Centralized System:

- non-autonomous components, autonomous means that all have to be up and running
- most of the time build using homogeneous technology
- Multiple users shares the same resources all times
- Single point of control and of failure

Distributed Systems:

- Autonomous components
- Heterogeneous technology (server linux, client android ...)
- components can be used exclusively
- executed in concurrent processes
- multiple point of failure, more complex but we have several nodes

The simple architecture of a distyributed system is a collection of processors having a non-shared local (main) memory exchanging message over a network. In case of failure we have a copy also of the ram in other device, in centralized system this is not possible.



0.6 Definitions

We need more definitions because there are different way of seeing the distributed system.

"A distributed system is one in which hardware or software components located at **networked** computers communicate and coordinate their actions only by **message passing**."

In this case we can see that the networking part is fundamental (located at networked) and then coordination among computational process by message passing.

"A distributed system is a collection of **independent** computers **that appear** to the users of the system as a single computer."

Then we have the Tanenbaum definition, here we have a more much abstract view, the distributed system have to appear like a single CPU even if it's spread across multiple CPU.

"A distributed system is one on which I cannot get any work done because some machine I have never heard of has crashed"

We have another one, maybe the most famous one, ideally is fine but the fact is problematical. If the system is configured correctly we have another one but it's not obvious. In the cloud is almost impossible to know where the problem are.

In some situation we can not avoid to have distributed system, Netflix must be a distributed system. Another trick is how to build a global distributed system without break some law, so in some case the distributed must be confined in local state.

Problem of deleting data in distributed system, sometimes data are impossible to delete.

0.7 Importance of the network

Every distributed system relies on a network service to communicate and it's one of the crucial point of failure.

0.8 Era of distributed systems

In the era of distributed system currently we discovered that GPU is a kind of CPU, even better for some activities (mathematical), GPU are great for parallelizing work like machine learning model. We start having framework for using GPU to do normal work not only for graphical work.

Also we have internet everywhere, big bandwidth.

0.9 Why we want to have distributed system?

We can have functional separation:

- Computer with different capabilities and purpose:
 - Clients and servers
 - Data collection and data processing

Inherent distribution:

- Information: nowadays is distributed by definition, created and maintained in different locations and media format (even our identity is distributed). We want to process data when they are collected, for example do some computation in the antenna and not store petabyte of data and then try to do some computation on that.
- Different teams work on the same functionality and they are separated autonomous entities on the network

Power imbalance and load variation:

- Distribute computational load among different computers (cluster), not all computer have the same computational capability

Reliability:

- Long term preservation and data backup at different location (replication)

Economies:

- Sharing a printer by many users and reduce the cost of ownership
- Building a super computer out of a network of computers
- Green IT, for example power consumption, way of cooling data center over a different PC over the world (millions of bottles of water per hour for cooling data center of chat GPT)

The very last definition: "A distributed system is a collection of autonomous computing elements that appears to its users as a single coherent system."

Autonomous system means that are autonomous in the sense that they can boot and stay up.

0.10 Characteristics

1. Collection of autonomous computing elements

- Referred as nodes
- HW or processes

2. Single coherent system

- Users or applications perceive single system
- Nodes that collaborates

0.10.1 Collection of autonomous computing elements

Every PC, nodes, must be autonomous and they have own notion of time (no synchronized clock).

There is a collection of nodes, several nodes that work together. But there are groups:

- Open group: any node is allowed to join the distributed system -> message to any other node
- Closed group: only the members of that group can join

The overlay network is what is important. There are different types of overlay:

- Structured: peer to peer, each node has a well-defined set of neighbours with whom it can
- Unstructured: each node has references to randomly selected other nodes from the system

0.10.2 Single coherent system

The system has to give the same response from the same request (coherent system), no matter where, when, and how interaction between a user and the system takes place. Distribution transparency:

- Impossible to tell where the computation is taking place
- Data location is irrelevant
- Replication of data is hidden

Resilience to failure of part of the system is not trivial, any internal failure should be masked.

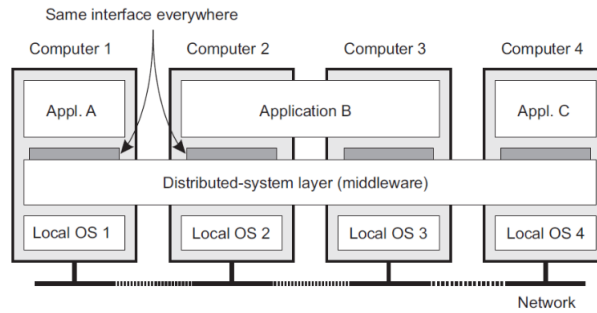
0.11 Middleware

How can we have a single coherent system when everything is distributed? Because it is shared how the system works, it's what is called middleware, it's a kind of operating system in the distributed system that allows us to add a node and make the things working. It's very important in fully distributed systems. We can also have a kind of middleware as an agreement of technologies (using docker, kafka ...).

It makes decision among technologies and framework to be used. It's common to OS that allow to use different components of PC the middleware is the OS for the distributed system, the difference is the services are offered through the network.

Commonly used components:

- Communication: how to set up the messages exchanged
- Transaction: set of instructions that must be executed in one time (DB transaction), executions in an all-or-nothing fashion



- Reliability: enhanced function for building reliable distributed application
- Service composition: we have to compose different functionality (web pages that combine and aggregate data from different sources)

0.12 Design goals

Transparency, but it's impossible to hide everything, for example we have some latency, it's not always for the network but maybe is for some services (disk error, computational error...).

Also failure in some situation can't be masked, sometimes we have to reboot.

So we have different degree of transparency, so let's try to see the degree of transparency:

- Access: we must make transparent the access to data, even if they are from different DB
- Relocation: Amazon put unused vm in old machine

Transparency	Description
Access	Hide differences in data representation and how an object is accessed
Location	Hide where an object is located
Relocation	Hide that an object may be moved to another location while in use
Migration	Hide that an object may move to another location
Replication	Hide that an object is replicated
Concurrency	Hide that an object may be shared by several independent users
Failure	Hide the failure and recovery of an object

We also have other goals: Openness:

- Interoperable, composable and extendable. We have tons of technologies that are open and are spin-off of something else (like Netflix). Open group maintains that kind of technologies.
- Open node group with policy to how to be part of the group

- Separating policy from mechanism:
 - Internal components and API vs monolithic approach
 - Many configuration to andle
 - Caching of browsing (mechanism) different parameters (policy)

Scalability, means in case of need I can scale:

- Number of users and/or process (size) -¿ computation, storage and network
- Geographical scalability -¿ reliability of the network
- Administrative domains

0.12.1 Scalability

In terms of size we can scale adding HW basically because the server, or groupof servers, canbecome a bottleneck when it needs to process an increasing number of requests.

Geographical means location, we can have some issues as:

- Latency of communications
- Reliability
- Very limited facilities for multipoint communication

Administrative: distributed systems spanning multiple administrative domains (federations of administrative organization).

That could be two different scaling:

- Scaling up: improving capacity (by upgrade CPU, increasing memory ...)
- Scaling out: expanding the distributed system by essentially deplying more machines

0.12.2 Replication issues

It's not just replication of data but also replication of capabilities and process.

Having multiple copies leads to inconsistencies:

- modifying one copu only will make the system inconsistent
- keeping copies consistent requires global synchronization
- Tolerating inconsistencies is application dependent

False assumptions in design:

- The network is reliable
- The network is secure
- The network is homogeneous

- The topology does not change
- Latency is zero
- Bandwidth is infinite
- Transport cost is zero
- One single administrator

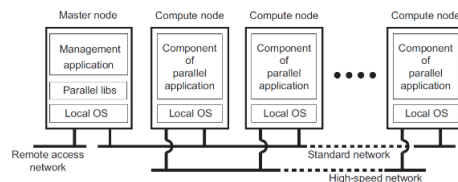
0.13 Types of distributed systems

- Distributed computing systems
- Distributed information system
- pervasive system: spread, like domotic and sorvelienace

0.13.1 Distributed computing system

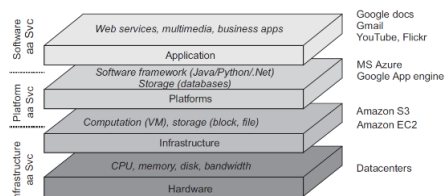
It's not geographical separation but it's a cluster with similar HR and OS connected with local network.

Then there are grid computing that are a federation of compute system with different HW and OS and maybe different location.



Cloud computing

Extension of Grid aiming to be open to external user. Utility computing: pay per use. Also is easy to access.



0.13.2 Distributed information system

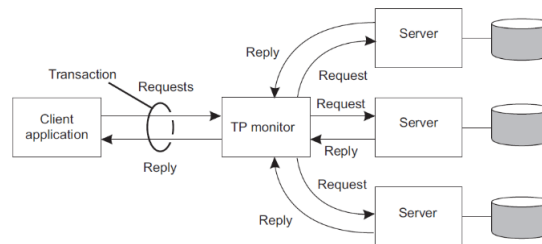
- Interoperability of many networking applications
- Network based client-server approach
- Enterprise Application Integration (EAI)
- Distributed transaction process: the key idea is that all, or non of the requests are executed

Nested transaction

In distributed systems, transactions are often constructed as a number of subtransactions, jointly forming a nested transaction, top level transaction fork off children that run in parallel and keep forking.

Provide a natural way of distributing a transaction across multiple machines.

We need a broker in between to take care of transaction to prevent infinite update and coordinate the execution of the transaction:



Middleware in Enterprise Application Integration

It could be:

- File transfer: Integration via transfer of file of shared data
- Shared Database: Application access the same database
- Remote Procedure Call: focus on actions and not on data
- Messagging: Decoupling of time and space

0.13.3 Pervasive system

Nodes are small, mobile and often embedded in a larger system, characterized by the fact that the system naturally blends into the user's environment.

- Ubiquitous computing
- Mobile computing
- Sensor network

0.14 Architectures

- Software Architecture: logical organization of a distributed system into software components
 - Components with well-defined interfaces. There are at least 2 important properties encapsulation and the fact that we provide only the functionality but not how they are implemented.
 - The data that components (form of the data) exchange must be compatible, if we want to pass some structure data we must tell how the structure is build
- System Architecture: final instantiation of software architecture. Maybe we can have a distributed software but centralized system.

Well-know styles for software architecture:

- Layered architectures: several layers like in software. Like cloud or virtualization. No way to go throught a layer without calling something on that layer.
- Object-based architectures is quite different because we have object and not layers
- Resource.centered architectures: the data is the most important thing. We are accessing in the same way to the same data that are shared and different actors will modify that data. Nothing is exchanged.
- Event-based architectures: the trigger is an event, the start of interaction is not a call from on other component but is an event. Usually we have sensor that start sending data (it could be an event)

0.14.1 Layered architecture

We have different layers, we can make downcall to component at a lower-layer and generally expects a response. In the other way we can have upcall like OSI stack. It's not used One thing that is very interection is that in each layer we have an ubstraction of the implementation of functions, one layer just use the call that is offered by the other layers.

Noramly this is a request response call.

Application layering:

- Application-interface layer: units for interfacing to the user
- Processing layer
- Data layer

It's also called Model View Controller.

0.14.2 Object-based architectures

Components are objects, connected to each other through procedure calls. It's not very used in distributed system because is quite complex, we have different process that want communicate on network, but network hasn't zero latency, especially if the processes are placed on different machines.

It relies in encapsulation, object can be stored and executed anywhere and anyway.

There are several technologies between object distribution, we have to expose the method of the object, we expose the interface.

Remote procedure call, is when we want to call one function but the function is not in the same machine, all the machines must allow to call something that is remote as if it's local this activities to calling other functions completed the procedure call engine. We have the remote object and our client (let's go back in client-server architecture, that is a distributed system), we have a client and each client must call something on the server as the server is local, we have proxy that impersonate the server.

This activity implies to know 2 concepts:

- Marshal
- Unmarshal

we need to have at least the interface of the function that we want to call replicated on the client, after that calling the marshal process starting and transform the calling in a message over the network.

It means that in case we move the server in other location we don't need to do anything because the remote procedure call does everything for us.

Normally this is based on OOP, the state of the object is server side, client only has the state of the client's objects.

This is also called RMI client and server, remember that we need remote proxy installed on client and server.

We cannot share the state of the objects because the state is in RAM and we cannot share the RAM between different nodes by a network, this is also the difference between distributed system and parallelized computers (they can share RAM). Interfaces are really distributed.

The architecture of this system is Service-oriented architecture (SOA): a distributed application or system constructed as a composition of many different services.

0.14.3 Resource-based architecture

Data is the most important thing. More recently this concept was developed in the concept of REST interface, is based on the notion of the resources. It's a kind of way to overcome the problem of the state of object, everything is data and we modify the data. We no longer need to expose an interface but only the data. In this case we have 4 very simple functions:

- PUT
- GET

- DELETE
- POST: modifying resource by transferring new state

This is familiar to functional programming, in this paradigm we don't modify anything but we create new and destroy the previous one.

By the way if we think about any computer programming the goal of each instruction is to modify the state (memory).

0.14.4 Publish-subscribe architectures

The fact that with REST we can't have the state of the previous calls is not always the best thing. We are able to do the stateless thing but the way they are calling is not a direct call but is a call by an intermediate. In order that the computers are in a different place usually all the pc must be up and running to call the functions, in this case we can separate the calling and the process of the data. This is a fully coordinated system.

2 different type of coordination:

- Referential coupling: a process can communicate only if it knows the name or identifier
- Temporal coupling: both have to be up and running

We can have different coppie di queste situazioni:

- Temporally and referentially: Direct coordination, they are up and running and know each other, we have direct communication
- Temporally decoupled, but referentially coupled: like mailbox coordination
- Referentially decoupled and temporally coupled: event-based coordination, they are up but they are triggered by an event, usually there is a broker that publish a notification
- Referentially and temporally decoupled: this is normally called shared data space. The communication is much more complex than normal shared DB, in this case we are sharing data for communication purposes.

Apparently this is the technology that is more in line with the definition of distributed system. In the real world we need to mix and decide what to do in the real situation. Being capable to decide which architecture implement that suit better with our situation is crucial, if we go for a wrong architecture is a nightmare to go back.

0.14.5 Event-based

In this case we have an event bus that provide events, each component must subscribe to that bus. The notification of event trigger bunch of activities on the component, for example a sensor can provide the data and the event that tell that data are ready. Each component completely ignore themselves and don't know everything about each other.

In event-based the event aren't stored, so if one component is not up and running the event is lose. In the other way in the shared data space we can handle a kind of discrepace, we can handle some kind of buffer.

In this case we have to publish a subscription that is needs to be passed to the middleware containing a description of the event that the subscriber is intresetd in.

There is a layer of software in the middle of component that allow us to implement that architecture (remote procedure call, public subscribe), it's not ad OS, it's framework a mix of different solution used in different context (set of different products togheter building the middleware).

Middleware organization

We have to create a distributed system putting toghether different tecnologies, but there are some patterns, like in computer programming.

- Wrappers (or adapters): eventually we have component speaking different language or using different protocols so we need to provide a correct interaction with them. Wrapping that specifi system in order to do transformations. The wrappers are different than interceptions, the system is fully wrapped means that in the rest of the world you can't see inside the wrapper. Nowedays adapters are moving outside the wrapper, we're thinking about adapters like other components. It's called 1 to 1 communication and is used also for make a common system a service in this case we wrap a program in a service in order to recall the program.
- Interceptors

Interceptors to be implemented is like a nightmare, but in some scenario is much more usefull than a wrapper. It's a kind of software that is in between break the normal flow of control and allow other (application) code to be executed.

0.15 System architectures

Where the software components are deployed. That's less important in the design point of view but is important in operational point of view.

We have system components, middleware and have to decide where to put all of this in order to have some level of performance.

We can decide to go for a centralized architecture, client server architecture:

- Servers offers, clients use
- Servers and clients in different machine
- Request/reply model
- A client could be a server for other client and also a server could be a client
- Organized in a central way

0.15.1 Centralized organization

We can go for different solution and we may need to start think about phisical things (how many node, how they are structured, OS, virtual or not). Imagene that we have multitire architecture (multiple nodes), in the example we have 3 layers we have to decide where to put the layers.

To the left to the right.

- Like in normal app we can use user interface on app but also, for example we can register, we can use the user interface on the server
- the user interface on the client and data and application on the database, we have to do downcall throught the network. It's similar to web browsing when we have fully static page, asking for a page through HTTP.
- In same cases we can have part of the application on the client and part on the server, UI on client and DB on server. Similar to dinamic web pages
- UI and application only client and data in the server, install application on the client and thell application that data is there.
- UI, application and a part of DB on client and other part of DB in server. It's similar to web pages with caches.

We have other example, multitiered blient-server. In some simple case, distributed processing

0.15.2 Distribution Model

In case of distributed model we have to consier 2 type of distributione:

- Vertical distribution: dividing distributed applications into logical layers, and running the comonents from each leayer on a different server
- Horizontal distribution: a client or server can be physically split up into logically equivalent parts. Each part is operating on tis own share of the complete data set (balancing the load)

One very simple scenario is peer-to-peer system, is an example of horizontal distribution:

- Structured: provide some structure, there is a super peer
- Unstructured: each peer is able to know the neighborhood and exchange message without knowing all the network exchanging messages using random walk algorithms
- Hierarchycally organized:

0.15.3 Hybrid Architectures

Combine client server, with decentralized architectures:

- Edge-server architecture: deployed on the Internet where server are placed at the edge of the network

Collaborative distributed system, (bitTorrent) for instance if we want to download the file each nodes have some chunks of files, we go to the node for the chunk. (not fully peer-to-peer)

Central system architecture is also a distributed system, only the deployment is centralized. The opposite of distributed system is monolithic system.

We have the network file system, is based on remote procedure call (RPC) and the notion of virtual file system, when we build a file system our node is collegato at one file system that only partially is local, the non local one is fully transparent.

An other example is the web browsing, even if it's simple is a distributed system. Is a multi tier in case of application server and service involved, if server uses different application to serve the page. Like in the example we have client, application server, DB server.

0.16 Process

Most of the concepts about processing is exactly system to OS course when we have threads, process, multiprocessor and stuff like that, we put emphasis on distributed system. Distributed system is not only distribute data, but is also distribute processor.

Nowadays the idea is to have layer that handle the request of the process in specific way. This architecture is based on processing manager and workers. If we want to call a function we give the function to a worker and the worker give us the data. Container is a worker, and composition of container is composition of worker.

Introduction to threads:

- Virtual processors, is software processors, not physical. But it's a software processors on top of physical one to execute programs
- Process: big entity
- Thread: part of a process, avoid blocking effect

Context are normally within the CPU, it's values in register, in some cases memory. So how can i move context? it's similar to context switching but it's something that we want to avoid, that's because we want multi thread, each thread have one context and we don't need to switch.

In DS threads are used mostly on the client but also in server side in case we want a multithread server dispatching the work. The middleware is a kind of OS so he has to take care of the common things of OS, also handle the threads. Dispatcher is a middleware that take care of threads. An example is APACHI Spark, is a middleware on top of the system which is capable to do big data analytics offering a fantastic way to dispatching work. The idea is to use RAM but if we have nodes we cannot share RAM, so they have

a very efficient dispatcher that receive data that decide who will do what, allocate the job on that node parallelize the job there. It's efficient but we cannot do this with a TB of memory. We aren't able to create a distributed system that go fast and use low memory.

0.16.1 Introduction to virtualization

From the point of view of performance this is not good at all, but to a point of view of design is the best, we can imagine to have a ton of nodes in the same machine, virtual nodes.

We have the possibility to virtualize much number of nodes and we can mimic a system that is not the one we have on our machine. Virtualization will help device software design and system architecture because of making physical things abstract.

We want to distribute processors, components, not virtual machines. The type of virtualization we want to achieve is application virtualization.

Different level of virtualization:

- Hosted virtualization:
- Native virtual machine we don't have the OS on machine but only the hypervisor

Client side transparency:

0.16.2 Server side

Stateless vs stateful, nowadays we are moving to stateless, like lambda function. But how is preserve de state? mostly of the process are stateless.

- Stateless servers: never keep accurate information about the status of a client
- Stateful server: keeps track of the status of its client, remember the previous interaction

Even if the server cannot maintain the state it delegate to the client to maintain the state.

0.16.3 Client-side: communication

We have the requirements of offering what is called software transparency, we offer the software in transparency way using what is called remote procedure call.

Client software includes more than just UI:

- Access transparency
- Location/migration transparency
- Replication transparency
- Failure transparency: we have more way to understand failure

0.17 Server-side

We have to implement different services for the clients. We can choose different type of servers:

- Concurrent server: uses a dispatcher, which picks up an incoming request that is then passed on to a separate process. Is the most complex way to develop a server.
- Iterative server: handles the request before attending a next request

We can also have:

- Stateless: never keep accurate information about the status of a client. There are possibility to keep information about previous interaction of client keeping information on the client (using cookies).
- Stateful server:

An other possibility is considering not server like a kind of monolithic but like a Object server when we have different objects that serve different function that response to different request. We can change services by adding removing objects. There is a way to scale a little bit. We can also have a replica of objects. The most important part is the activation policy: is how to invoke a object. We can use an object adapter that wrap a group of objects by policy.

Then we may have server cluster. Could be:

- Local: are the larged use in the company. We can distribute servers in the same building. This because of low latency, in distributed system we can have large latency, because if we make a request we can't wait forever for the response blocking all the system.
- Wide
- Organized into tiers

0.18 Migration

Distribution is not just data passing, but also process passing. Imagine something that process parallelize, the code is always the same, we just have to distribute the code. On top of this we have to put an other job that collect the result.

In other way we could have the need to move computation for balance the load on machines.

For ex. we have a client-server system where server manages a huge database. The client application involves large quantities of data. The idea is to put some computation on server side, close to the data.

0.19 Communication

We have the OSI layers. The middleware is implemented as a layer, it provide common services and protocols that can be used by many different applications.

- A rich set of communication procotols
- marshaling of data, necessary for integrate system
- Naming protocol, to allow easy sharing af resources
- Seurity procotol
- Scaling mechanisms

On top of this we have the application layer:

- DNS is application level in terms of OSI but... it is application-independent and is used in the middleware.

0.19.1 Protocols

A process must be atomic, means that all processes carry out a particular operation, or the operation is not carried out at all.

0.19.2 Types of communications

- Transient vs persistent
 - Transient: server discards message when it cannot be delivered at the next server, or receiver
 - Persistent: a message is stored at a communication server as long as it takes to deliver it
- Asynchronous vs synchronous
 - Asynchronous: sender continues immediately after
 - Synchronization: in distributed system is very complex, because we don't have common clock. We have to wait untile something come back.

Client/Server computing is generally based on a model of transient synchronous communication because client and server are acvite at time of communication, client issues request and blocks until it recived reply. Server waits for incoming requests, and subsequently process them.

Message-oriented middleware aims at hig-level persisten synchronous communication: process send each other messages, which are queued. Sender need not wait for imediate reply, but can do other things. Middleware often ensures fault tolerance. Fine having microservice, but always we have some drawbacks, we miss completley the control and we are not able to debugging the code.

0.19.3 Remote Procedure Call

The important thing is what is the call.

We have different parameters and different semantics. Copy in/out semantics: while procedure is executed, nothing can be assumed about parameter values. That's the way the normal procedure works.

0.19.4 Distributed Computing Environment

We have some distributed system that focus on computing. Most of them are based on RPC, there is a middleware system executing in between existing operating systems and distributed applications.

0.19.5 Message-oriented communication

Advanced transient messaging.

in this case we also have the broker in between in order to handle application heterogeneity. Maybe it can modify the messages to the target format.

0.19.6 Multicast

It's the simplest one. We send data to multiple receivers.

At application layers we have different way to do multicast:

- Use overlay network to disseminate data
- Tree: leading to unique paths
- Mesh networks: requiring a form of routing

An other much simpler way of dissemination of request is flooding based multicast. There are also the gossip based data dissemination, we pass the information to the neighborhood.

0.20 Naming

Naming is crucial, we have a lot of entities in distributed system and we have to reference to them. Apparently it seems to be easy, but there are several ways to achieve this. The easiest way is to give an identifier, this identifier could be human-friendly or not. In some scenario is better to have human readable name (registrationServer...). We can have structured names that refer to cluster of servers. We can also have attributed based naming.

We can have hierarchical structure and hierarchical way to go through the names.

There are several ways to resolve names, usually we have DNS layer, but we can also have broadcasting or multicasting communication. For instance DNS is hierarchical structure.

Flat names are good for machines but not for humans. Structured names: organized as name space:

- A naming graph in which a node represents a named entity and the label

Attribute-based: name as pairs (attribute, value). It offers an unique and location-independent way of referring to entities.

0.21 Coordination

How process in distributed systems synchronize, there is no notion of globally shared clock. The important thing is preserve ordering of calls, not strict synchronization. For instance distributed mutual exclusion algorithm ensure that in a distributed collection of processes, at most time

0.22 Replication and Performance

Data are generally replicated to enhance reliability or improve performance. The main issue is ensure that all conflicting operations are done in the same order everywhere. Replicate data is quite complex, because we have to keep persistency on data. If a data is modified on node B node A need the updated data.

Imagine we have a data-centric models, in which we store specific data and do read and write to that data. The fact we cannot really work of the idea of having all the consistency all the time, we need to consider the system some degree of consistency. We cannot consider the replica consistent all the time. So replica may differ in their numerical values and may differ in their relative staleness. There may be differences with respect to the performed update operations but bounded. To define inconsistency a consistency unit has been introduced, specifies the data unit over which consistency is to be measured, we keep consistency in just part of dataset.

Sequential consistency: the important thing is the sequential order. The result of any execution is the same as if the operations of all processes on the data store were executed in some sequential order, and the operations of each individual process appear in this sequence in the order specified by its program.

There is an other consistency which is important, is the causal consistency: writes that are potentially causally related must be seen by all processes in the same order. Concurrent writes may be seen in a different order by different processes.

0.23 Fault Tolerance

- We may have partial failure, part of the system may fail and the other part keep working. The system seems to be working.

0.23.1 Dependability

A component provides services to clients. It may require the services from other components.

Chapter 1

Virtualization

Virtualization is a building block for cloud computing. The other building block for cloud computing is computer network.

In computer science virtualization is used to indicate what is not real. It's very important that simulation, emulated and virtualization are not synonymous.

1.1 Definitions

Activity aiming to create replacements (virtual resources) for real resources, which have the same functionalities and external interfaces of their counterpart, but different attributes (dimension, performance, cost). A mechanism through which virtual version of resources, usually provided physically, are being created. A technique used to recreate - through software - an environment that looks like a hardware to the host operating system.

Remember that full transparency never achieved because we deploy resources through the network.

1.2 Emulation

Virtualization is a concept that was created in '60. Emulation means that we take a classic PC and we run a software that is not intended to be run on this architecture. So basically we take the software as it is and run in different architecture. We have an emulator software that emulates the hardware.

In this case we are translating a set of instructions for system A to a set of instructions for system B. This is very consuming, latency is very high.

We use emulation because we want to run a software that is not compatible for our device. Or windows to mac stuff. Reading data written on a memorization device through a device that we no longer have or that no longer works.

1.3 Simulation

Simulation is faster but less precise and slow. Means that we are not going to take the original software but we rewrite the software for the new architecture.

There is also high level simulation

1.4 Virtualization

Virtualization is the technique for using resources and devices in a functional way, without considering their physical layout. The detaching of data from computers are the base for cloud computing and GDPR.

Including the division of a single physical computer in many virtual machines with a dedicated hardware. So the idea for the user is to run a completely physical machine. The trend is moving from a physical resources to a virtual resources.

- Each machine has a full and dedicated environment (encapsulation)

- Each machine is isolated from each other just like physical separation (isolation)
- Independent from underlying hardware (hardware independence)
- Created using existing hardware (partitioning)

We use a virtualization layer that allow us to do all this stuff.

Reason for virtualization, issues:

- Too many servers have a small workloads, the same happens with our laptop. For our laptops is ok but it is not in business.
- Old hardware does not work. When we have big data center we have to maintain it. When we use provider for HW we don't need to handle HW obsolescence.
- Infrastructural requirements are always increasing
- Small flexibility in shared enviornments. If we have physical enviornment it is less flexible.
- With physical architecture we have a lot of costs like Maintenance, Leases, Networking, Floor space, Cooling, Power, Disaster Recovery and so on.
- Also we have heterogeneous enviornments (Linux, Microsoft, IBM, Apple, SUN)

Advantageses for virtualization:

- Consolidation and hardware costs decreasing. Here the key point is decrease the usage of HW and decrease the cost of business. Doing this we have also a green effect, on a long therm we are decreasing the amount of the physical machine and resources.
- Workload optimisation: virtualization enabled dynamically answering to applications needs. We can move our resources back and fowrth virtual machines. It's also possible to use virtualization to increase resource usage, enabling dynamic sharing of resource pools.
- Flexibility and IT responsiveness: allows reducing the management of the enviornment, providing emulation to support compatibility and increased interoperability.
- Better performance
- Transparency
- Heterogeneity
- Portability
- Interoperability
- Green IT

Some scenarios:

- Server consolidation: many network services offered by distinct servers are migrated into a single server
- Testing: it is possible to duplicate a test server into a production one
- Training: it is possible to provide a complete study environment, which is hardware-independent

What we have to do for manage virtualization:

- Analysis and planning
- Adaptation and post-adaptation period
- Virtualized infrastructure maintenance

1.4.1 Analysis and planning

In this scenario we have some HW architecture and we want to study how to pass in a virtualization system. First of all we have to study the compatibility and support of existing hardware. Then we have to study the licenses that we have:

- Some software restrict the number of instances
- Other ones do not have restriction

We need to plan migration and understand if we'll experience some downtime, very often we have to be careful of the impact if we shut down the environment. Staff training. All of this are summarized in Return of Investment evaluation.

1.4.2 Adaptation and post-adaptation

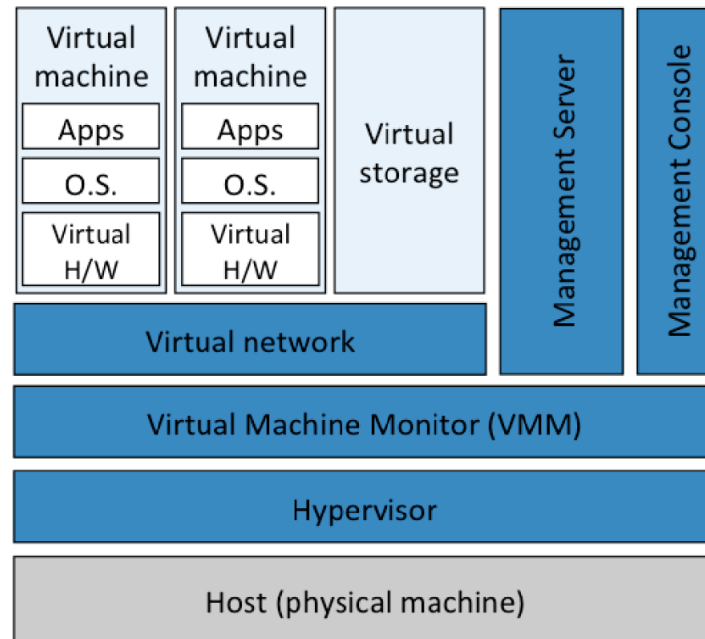
We have to consider about non functional aspect:

- Reliability
- Performance
- Efficiency
- Security

1.4.3 Maintenance

Once our system is working we have to maintain it.

- Scalability: hardware does not grow with infrastructure growing
- Security of virtual machine and virtualization platform
- Responsibility: roles and privileges
- Evaluation of the virtualization market



1.4.4 Terminology

- Hypervisor: the system component realizing virtualization. Allows multiple OS running on the same computer.
- Virtual Machine Monitor: the application component realizing virtualization. Is part of the hypervisor. Provide transparency.
- Guest OS, the virtualized system
- Host OS, the real system. Very often is just the hypervisor
- Management Server
- Management Console
- Network Component
- Virtualized storage

1.4.5 Virtualization means

he system executed in the virtual environment must behave exactly as it was executed on an equivalent physical machine. The virtualization environment must have full control over the virtual resources Transparency, full control and efficiency. A statistically relevant percentage of instructions must be executed without involving virtualization, most operation must be executed by physical HW and that make virtualization efficient. The challenging part was how to make virtualization efficient.

1.5 Popek and Goldberg conditions

Efficiency requirements are the starting points to determine the sufficient conditions on the hardware architecture

For any conventional third generation computer, a virtual machine monitor may be constructed (and is efficient) if the set of sensitive instructions for that computer is a subset of privileged instructions (trap in user mode).

The majority of instruction are executed natively on the HW. In other words: VMM is efficient if:

- All instructions that can alter the functioning of the system are executed in privileged mode, through an exception (resource control)
- All non-privileged instructions are natively executed (efficiency)

Classification of machine instruction:

- Privileged instructions: they raise an exception (trap or fault) if executed in user mode. Do not raise an exception if executed in kernel mode
- Sensitive instructions (control): try to modify the configurations of system resources (global system state). Change the control.
- Sensitive instructions (behavior): their result depends on the configuration of system resources (global system state). The instruction depends of the global state.

The Virtual Machine Monitor must satisfy three properties:

- Equivalence: the behavior of the Guest OS must be identical to the one of the host OS
- Resource control: VMM must have a complete control of all HW/SW resources of the host OS
- Efficiency: a significant rate of the guest instructions must be executed bare metal without the intervention of the VMM

The second theorem is defining the recursively virtualization. We can virtualize over virtualization if:

- Virtualizable in an efficient way
- A VMM without any timing dependencies can be constructed for it

Theorem 2 tells us when a hardware architecture can execute a VMM in a recursive way, a VMM executing VMM, which executes a VMM...

All virtualization approaches are built on Popek and Goldberg conditions.

The problem was that initial implementations of x86 did not meet Popek and Goldberg conditions, virtualization suddenly stop.

VMware relaxed the definition and 3 approaches were created:

- Dynamic binary translation (first)
- Hardware-assisted virtualization
- Paravirtualization

1.6 Taxonomy

- Full emulation
- Partial emulation

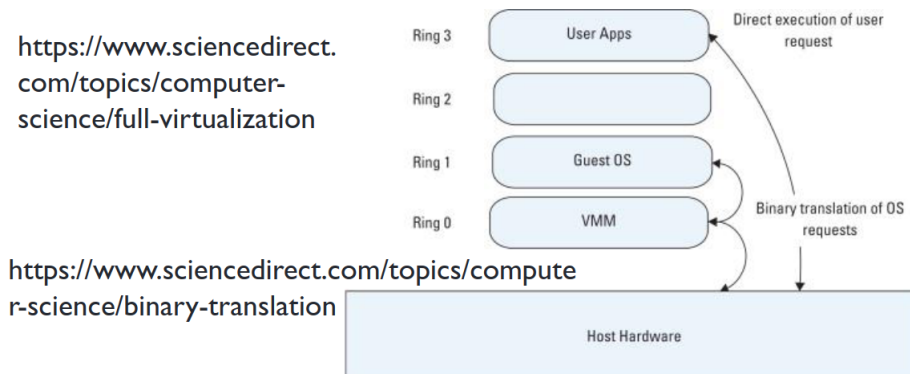
1.6.1 Full emulation

We have the complete virtualization of the HW and we emulate. We can run an unmodified guest OS on a completely different host architecture (different instruction set).

1.6.2 Full virtualization

VMM emulates only the necessary HW allowing the isolated execution of guest OS. Guest OS must be designed for the same CPU of the host OS.

So full virtualization using binary translation so the idea here that we translate the Guest kernel into code that could run directly and safely on the hardware. A binary translator replaces the sensitive instructions by equivalent non sensitive instructions at run-time, and leaves non-sensitive instructions unchanged.

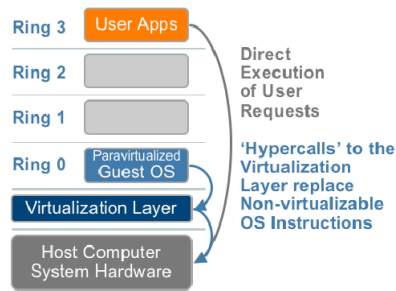


The instructions that produce problem in Pop e quell'altro condition are translated binary.

1.6.3 Paravirtualization

Is a scenario in which we don't have any emulation. The Guest OS must be extended in what we call hypercall, that is virtualized version of the system calls. Call service of VMM and is called by means of exception.

Here the goal is again make virtualizable the call that are not. Guest OS and hypervisor communicate to improve performance and efficiency. Modify the OS kernel to replace non-virtualizable instructions with hypercalls that communicate directly with the virtualization layer hypervisor. Hypercall interfaces for critical kernel operations such as memory management, interrupt handling and time keeping.

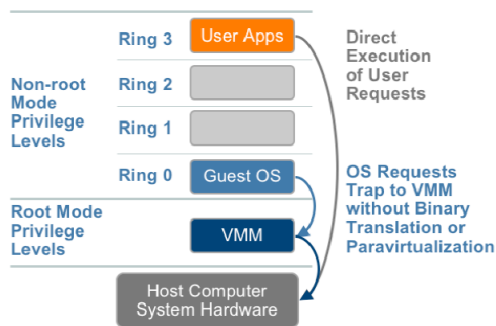


1.6.4 HW-assisted virtualization

Extend CPU with virtualization technology. VMM provides OS guest with true hardware through some processor extensions.

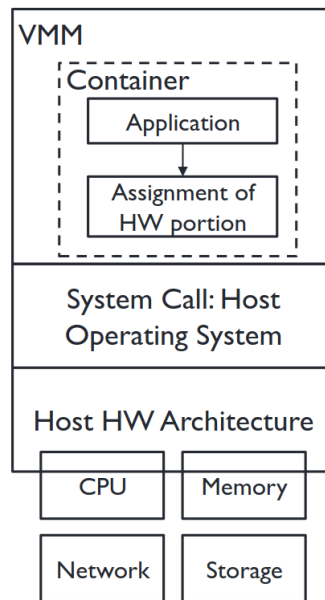
Provide OS Guest through true HW using processors extension.

A CPU execution targeting privileged instructions. Allows the VMM to run in a new root mode below ring 0. Privileged and sensitive calls automatically trap to the hypervisor.



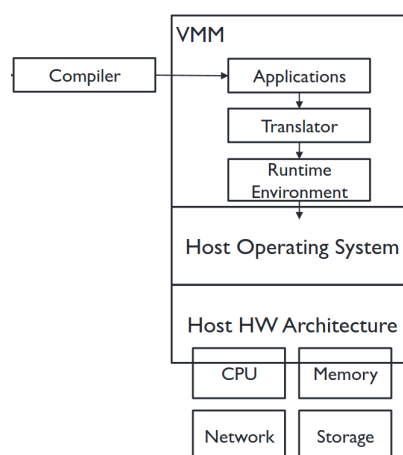
1.6.5 OS-level virtualization

The idea was to partition HW resources partitions hardware resources of host OS between the guests. Resources are partitioned in containers. Much more slimmer approach than emulation/simulation. It's possible to execute only guest OS with the same kernel of host OS

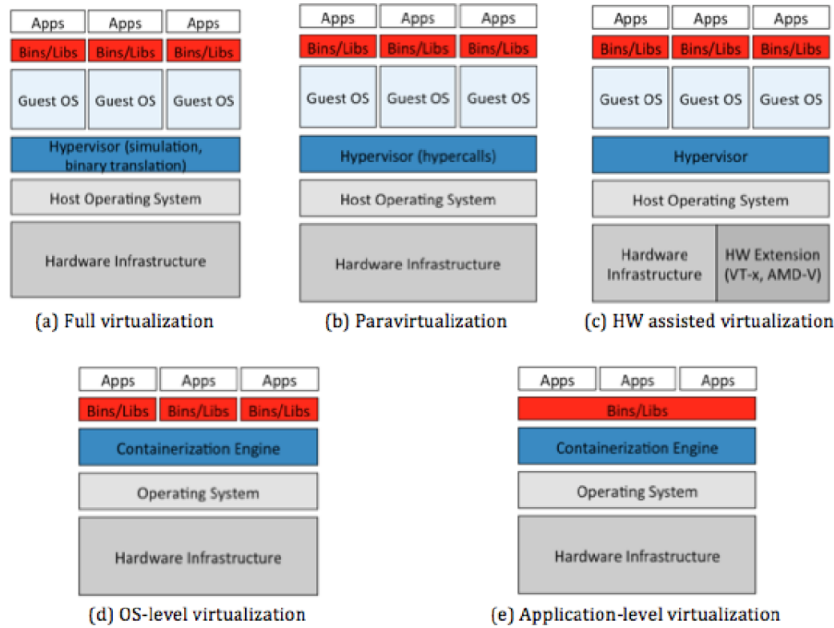


1.6.6 Application-level virtualization

Is Java virtual machine. We have compiler that produce bytecode and we can execute this bytecode in a virtualized machine and we can run this machine everywhere.



A recap of taxonomy:



1.6.7 Native and non-native virtualization

If a virtualization system expects the explicit use of a Host OS and Guest OS, it is a not-native virtualization. It's slow and OS and Guest cannot be different. VirtualBox is an example.

If a virtualization system expects the explicit use of a unique OS, it is a native virtualization. Fast, Guest and Host coincide in the same OS. We have bare-metal hypervisor.

1.7 VMware virtualization platform

vSphere is VMware's virtualization platform it's suite of virtualization solutions. It includes vCenter and ESXi. vCenter provides advanced server management capabilities through a centralized platform. Automate and deliver virtual infrastructures. Allows to manage ESXi hosts, as well as virtual machines up to 2,000 hosts and 35,000 virtual machines.

Mware vSphere Hypervisor (ESXi), a robust bare-metal hypervisor that installs directly onto your physical server. Gives the fundamentals (hypervisor) to build and manage a virtualized IT infrastructure. Abstract processors, memories, disks and network resources executing applications and unmodified OSs.

The hipervisor is really small, all the virtualization enviornment is installed direct on the HW and it's very slim and lighth. Have small architecture improve security, because we have less line of code.

It partitions the physical server into several virtual servers. Each virtual machine is a complete system (CPU, memory, network, storage, BIOS). Isolated virtual machines (high availability). Failures and configuration errors in a virtual machine does not influence the other ones

Performance and scalability close to real machine ones. We can configure virtual machines up to 128 virtual CPUs, 6 TB of RAM and 120 devices. Availability, security,

interoperability to provide adjunctive functionalities to software applications.

1.8 Xen

It's a paravirtualize hypervisor, it's open, it support different OS and Cloud platform. It can scale up to 4,095 host CPUs with 16Tb of RAM (physical):

- Using Para Virtualization (PV), the hypervisor supports a maximum of 512 VCPUs with 512Gb RAM per guest
- Using Hardware Virtualization (HVM), it supports a maximum of 128 VCPUs with 1Tb RAM per guest
- Reliable: over 10 million people use Xen with an ecosystem of more than 2000 commercially certified partners
- Flexible: architecture enables vendors to create Xen-based products and services for servers, cloud, desktop, embedded, and security-first environments
- Support: hypervisor supports multiple guest operating systems and cloud platforms
- Secure: Minimal attack surface with key security features like KCONFIG and Virtual Machine Introspection
- Modular: Architecture enables a degree of scalability, robustness, and security suitable for large, critical, and extremely secure environments

The crazy thing is that the hypervisor is made by less than 65k lines of code.

1.9 KVM

Is a full virtualization solution for Linux on x86 hardware. It's the virtualization used in open stack.

Can execute unmodified Linux or Windows images. KVM is open source software.

The kernel components of KVM are included in mainline Linux since version 2.6.20. They provide direct hardware access.

KVM userspace tools and components userspace di KVM are included in mainline QEMU since version 1.3. Userspace provides and exposes devices to the Guest OS. When the Guest has to emulate devices it goes through QEMU-kvm that is responsible for hardware emulation. When the guest has to do the real activity it speaks with kernel modules.

1.10 Virtual Desktop

The idea is to provide a desktop that is virtual, it can be done in different way. One old way was the remote connection, than we move on virtual machine, than we have the idea of virtual desktop infrastructure moving the complete infrastructure to a datacenter

and connect to that datacenter, that was possible for improvement of internet connection (strong local area network). The next step is cloud computing, the computer network is so huge that is possible to implement the same approach in wide area network.

It's also very convenient for:

- Centralized management and security: we have a single administrator
- Business Continuity
- Isolation and standard way of managing PCs
- Decreases the need to buy new hardware
- Decreases the time of adding a new image
- Centralized administration of all desktops, eventually located anywhere in the world

1.10.1 Single remote desktop

Team viewer, infrastructure that is build for control a PC through Internet connection.

1.10.2 Blade physical desktop

Buy physical PC and give the management of this PC to an authority. Users have their own PC, but the physical hardware is a blade PC located in the datacenter. Pros is that we delegate the maintenance of PC to other and cons is that we buy a complete server but we'll use a part of its computational power.

1.10.3 Virtual desktop infrastructure

Most similar one to the cloud approach. We have big server room in which we have Server Virtualization and a broker that provide access to virtual desktop to employee.

So we have centralized management (like in cloud computing), and we have dedicated images to users or a pool of standard images.

VDI Central Hosting

This really based on network. It requires a continuous network connection. A client connect to the gateway (switch or router) and through the gateway it reach the VDI broker, the broker authenticate the user, take the image to the image store and load the image to the virtual host. After this the hardware is ready to be used by final user.

The idea here is that the interaction is exactly the same like our OS is installed in our physical machine. In case of high latency we see that we are not using real HW. Nowadays we are increasing the reliability of the internet connection.

VDI Local Hosting

We have the gateway that contact the broker and the broker move the image of OS direct to our PC. In this case we need more than good internet connection. It's convenient only if we are in local network.

1.11 Server virtualization

We have a physical server at the bottom on we over impose a virtualization layer made by hypervisor and so one, and on this we build different virtual machines.

The difference of these virtual machines between classical virtual machines is that they have to serve services.

Every virtualized infrastructure permette to handle disaster through disaster recovery.

1.12 Data Center virtualization

The idea is to use IT like a commodity, like watching a movie.

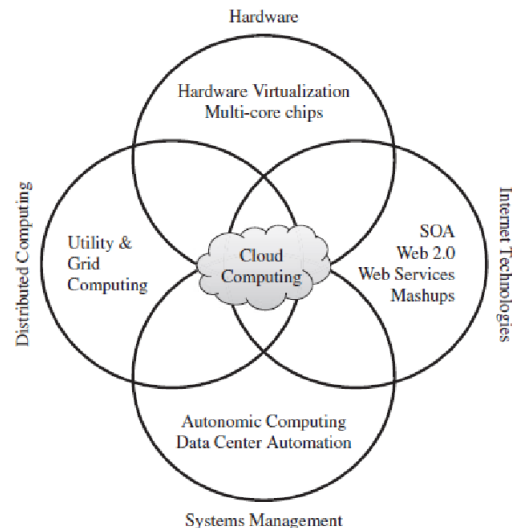
Chapter 2

Cloud

The goal is to make IT an utility, an utility is something that we pay per use, someone else manage for us the utility. Usually in cloud we have just a graphical interface and point and clicking we can create a machine, without knowing how the system is working.

We want to reduce as much as possible the interaction with human. The idea is monitoring the traffic of the machine and scale the capability when I need.

All this domains contribute to cloud computing:



Rajkumar Buyya, Andrzej M. Goscinski, Cloud Computing: Principles and Paradigms

We want to use computer like we are using electricity, we don't want to know how the infrastructure works, but we just want to use it.

In utility computing we have to match user desiderata and infrastructure capability. Service providers define utility in terms of profits.

In cloud computing we have also some aspect of autonomic computing, beacuse the system automatically adapt and scale.

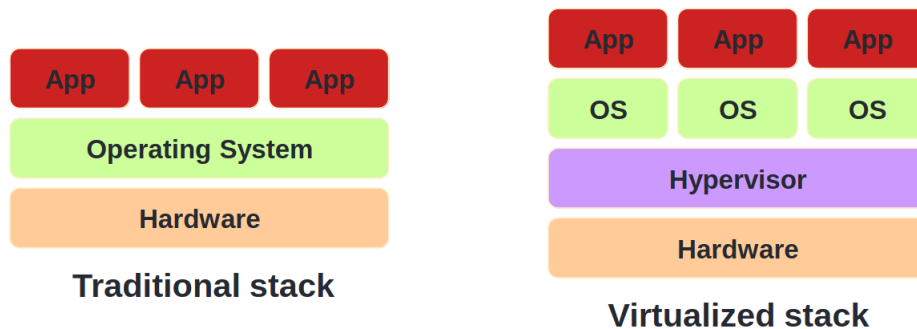
Cloud is based on three aspect:

- utility computing
- autonomic computing
- distributed systems

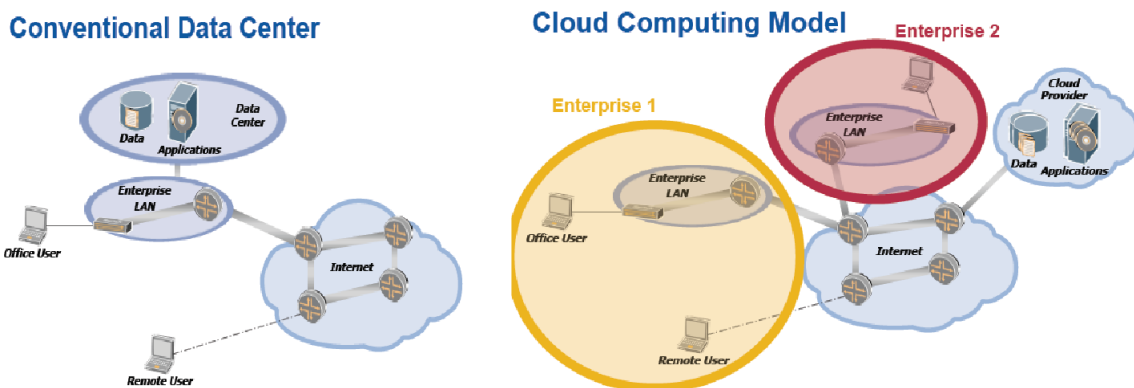
IT as a service:

- Sotrage, data processing and additional IT services distributed by external providers
- Application as computing utilities, no need to know the computing infrastructure nemeath
- Pay as you go

The key technology is the virtualization:



We pass from a private data-center to the cloud:



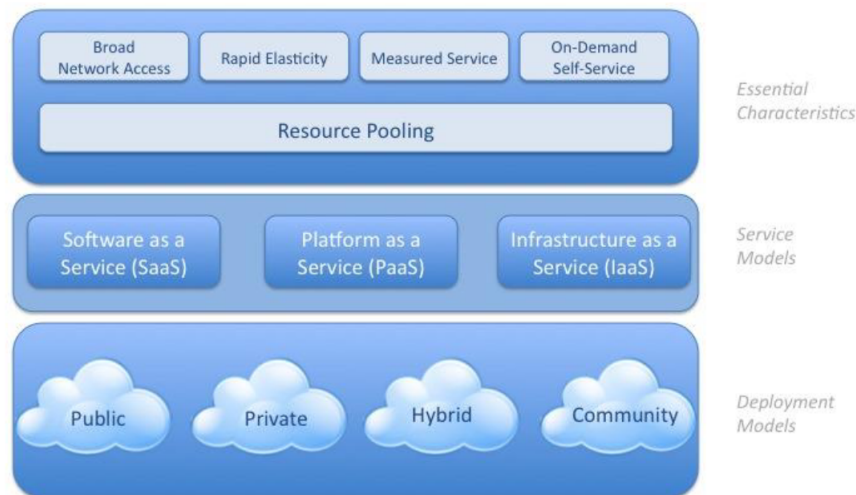
We have some problems about privacy, because is like we are sending our data to an external provider. But we are improving security because usually the provider is big and maybe is capable to counter the attack.

Also the GDPR regulation started with advent of cloud, because with cloud we don't really know where the data are stored.

Cloud computing is similar to electric power distribution. Concept extended to IT:

- Functionalities released hiding internal functioning
- Computers integrate distributed components providing processing
- On demand access to resources
- Pay per use
- Infrastructure is a black box
- Provide computing, storage, software as a service

Definition: "Cloud computing is a model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and



released with minimal management effort or service provider interaction. This cloud model is composed of five essential characteristics, three service models, and four deployment models”.

Main characteristics:

- On-demand self-service: a client can produce resources, server time and network storage on demand and interacting with service providers in an automatic way.
- Broad Network Access:
 - Resources available through the network (huge bandwidth)
 - Access by means of standard protocols
 - Support for all device types and platform
- Rapid elasticity:
 - Ability to scale resources in an elastic way based on real needs
 - Scale out, in, down
 - A client has the impression of having infinite resources, though it is not true, we still have HW and HW is finite. Also the money of client are limited.
 - No waste of resources typical of on-premise systems
- Measured service
 - Cloud manages and optimizes resources dynamically, using metering functionalities (pay per use)
 - Resource usage is monitored, controlled and logged providing transparency to cloud providers and customers. We don't have the same transparency that we have on our system.
- Resource pooling:

- Resources are divided to serve multiple clients using a multi-tenant model
- Physical and virtual resources are dynamically assigned on demand
- Location independence: client does not have control on the resource location, though it could require a specific location at different granularities. There is not direct link between the location of client and the location of client's assets.
- Resources include storage, processing, memory and network bandwidth

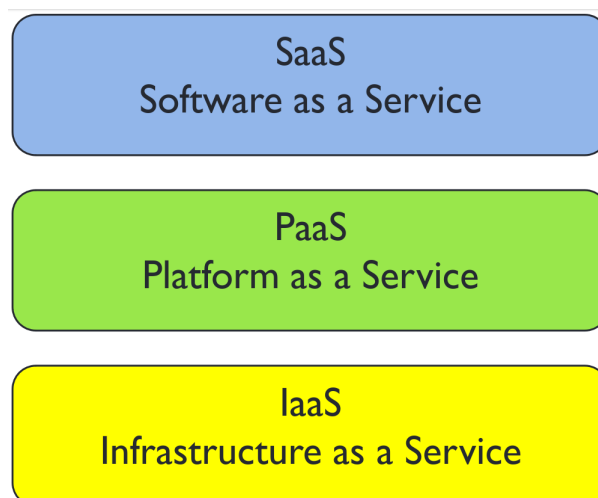
We have additional characteristics:

- Lower costs: we split the cost across several different users
- Ease of utilization: users do not have to be a computer scientist to run an infrastructure
- Quality of service: address expectations in the contract with the provider
- Reliability:
 - Provide scalability, load balancing, failover. BY DESIGN
 - More reliable infrastructure than the ones under direct control
- Outsourced IT management: User manages the business, someone else the computing infrastructure

Cloud computing is based on multi tenancy. A single software instance is shared by different enterprise/clients (these are tenants). User data are virtually isolated, physical isolation not implemented. Another critically important is how can we verify the activity on cloud machine? for security problems.

2.1 Service models

Which are the services that I can collect from cloud computing:



2.1.1 IaaS

Users manage the whole processing (CPU), memory, storage, network and additional computing resources. Is really similar to having a virtual machine, we take the virtual machine in the cloud and we install on top the services that we need.

Users can install and execute generic code including operating systems and applications. Users neither manage nor control the cloud infrastructure, while it controls operating systems, storage, and installed applications. We don't need to control HW so obsolescence is not longer our problem. Users has a limited control of network components (host firewall).

Provide virtualized resources on demand. Providedifferent serers with different operatign systems and an ad hoc stack software. Amazon provides virtual machines with different combinations of operatign systems:

- EC2 Service
- It is similar to having a physical server
- Users can start and stop a VM, install a software, connect virtual disks

Based upon the pay per use concept. I buy computing, storage, network resources and I pay for what I use:

- Amount of traffic generated
- Number of CPU cycles used (per second). They try to increase the precision in observing the usage to reduce the bill's amount.
- Amount of storage used

Advantages:

- Pay per use
- Modular scalability
- Security
- Reliability
- Access through APIs, that offer automation

An example of IaaS could be a temporary shop, we can do some example in computer science field:

- Recurrently execute a batch job without owning a proper machine for execution: use a virtual machine on Amazon EC2
- Activate a temporary Web site (few days): use a virtual web server on FlexiScale
- Provide employee with a remote storage without sufficient storage in the enterprise: use Amazon S3, it's similar to an external hard disk

2.1.2 PaaS

Was mostly thought for the developer, it makes the cloud programmable and provide to the user a platform ready to be used. As we know before starting programming we have to configure our ambient. Users do not manage or control the infrastructure and also the platform. We want an ambient that is ready to be used from developers.

Users keep control on installed platform, but as we said not of the infrastructure. Users keep control of installed applications and environment configurations for application hosting.

Paas is the abstraction layer making the cloud programmable. We have a platform installed on the infrastructure:

- Platform offers an environment where developers install and create their applications
- No need of knowing the infrastructure
- Multiple programming models and specialized service offered as building blocks

You develop the overall logic of application and integrate the common building blocks. In the end we pay for using this specific platform, in this case we pay for the platform and we receive also the infrastructure layer in which we execute the platform.

Paas sometimes is based upon a mix of usage and subscription model.

Advantages of PaaS:

- Services deployed on demand
- No a priori estimation of amount of requests, procurement, ...
- No management overhead
- Pay per use
- Modular scalability
- Security
- Reliability
- APIs

Main services:

- Libraries, tools and platforms for web development. They take care for us.
- Computing platform
- Mainly for programmers

Example

Problem: develop a web application or a cloud service without installing the whole software stack including libraries and tools.

We use Vercel to deploy our application. It's continuous integration and deployment.

2.1.3 SaaS

Is the idea of providing a software to the users. Users use the applications of a cloud provider executed on the cloud infrastructure (Gmail, Google docs...). Applications accessible by means of different client devices, for example web browser. Users do not manage or control the cloud infrastructure (including network, operating system, server, storage), and application-specific functionalities.

Applications are deployed at the top of the cloud stack. Services are accessible using a browser. Paradigm shift: from software installed locally to software installed remotely. Reduce the effort of the users in the application management, and simplify development and testing for providers. Salesforce.com is an Online CRM, on demand access and configuration of applications.

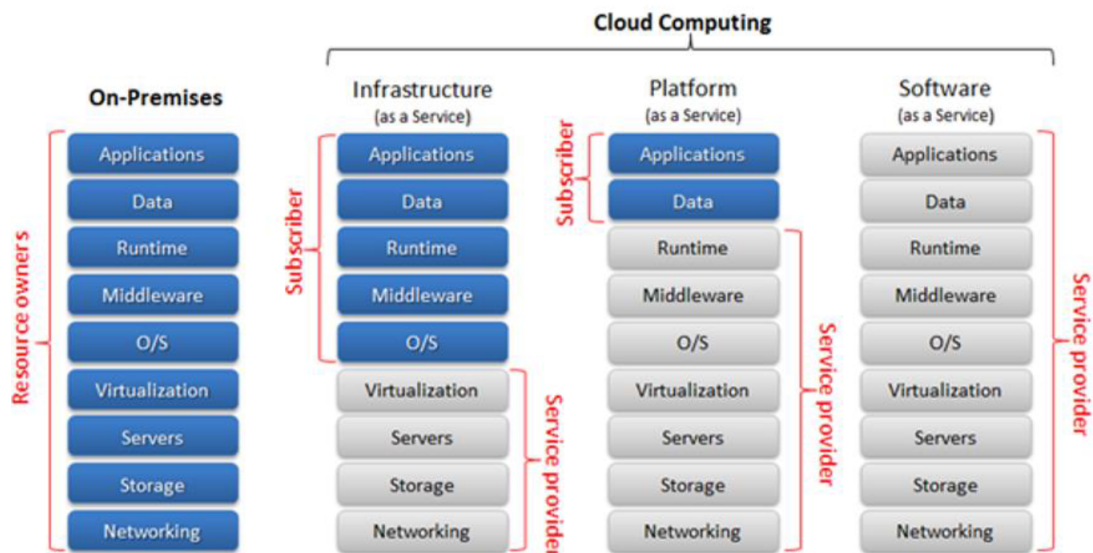
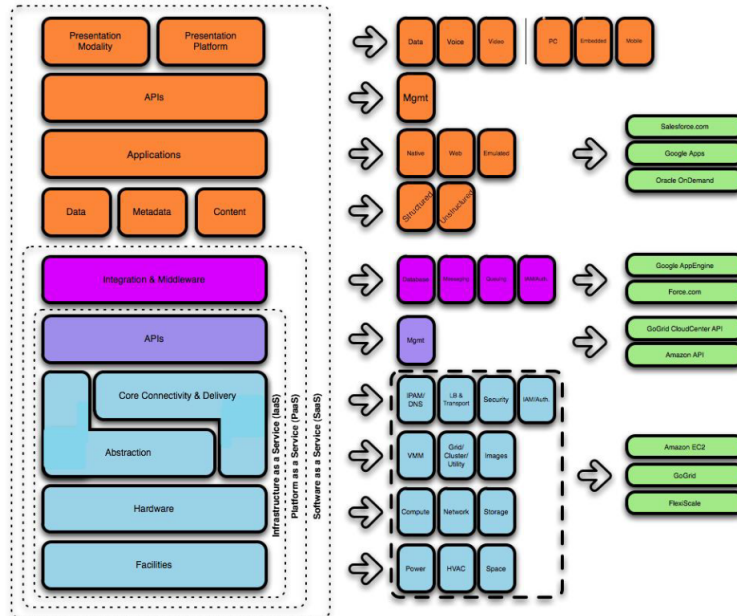
Advantages:

- Mainly for PMI
- No management of hardware/software
- Access via browser
- Pay per use
- Modular scalability
- Security
- Reliability
- APIs

2.1.4 Stack overview

Cloud Reference Model:

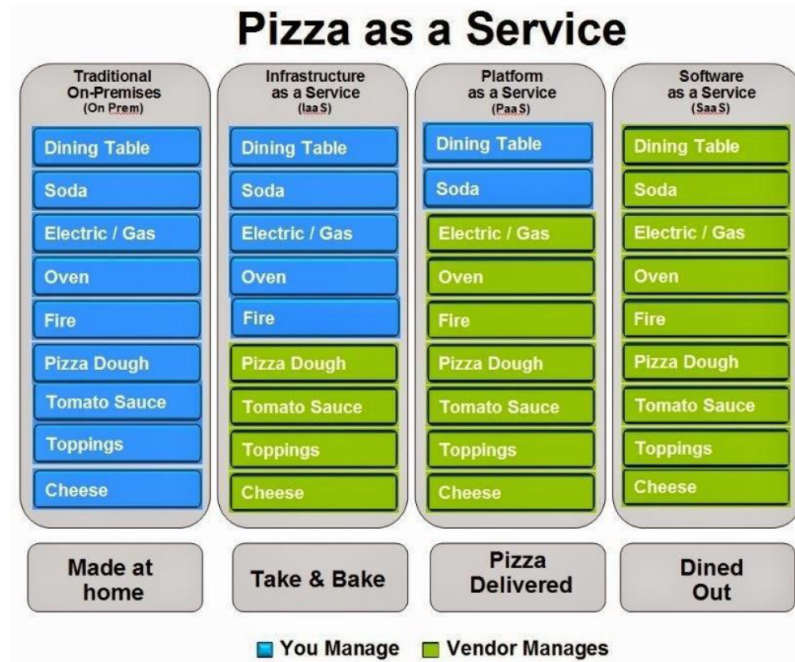
- The lower portion includes hardware and infrastructure (including network)
- Each model inherits the ability of the models beneath



Basically we can have everything like a service. We have several domains and every can be delivered as a service.

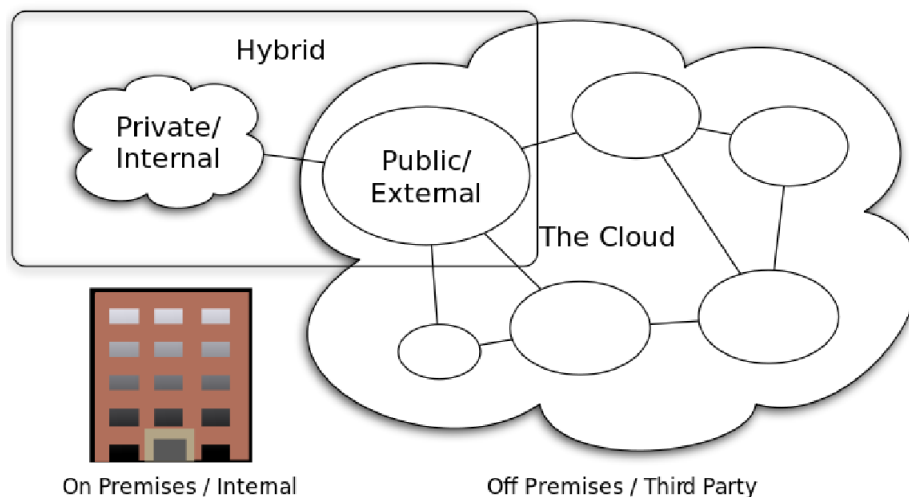
2.1.5 Deploy models

- Private cloud



- Community cloud
- Public cloud
- Hybrid cloud

Are just different approaches to deploy our business to the cloud.



Private cloud

Cloud infrastructure provided for an exclusive use of a single organization including multiple users (tenants), For instance, UNICloud. Owned and managed by a single organization, third party, or a combination of the two. Can be on premise or off premise

The difference is that that organization has to handle the HW.

Community Cloud

Cloud infrastructure provided for an exclusive use of a community of users. Users include organizations that have common goals (e.g., mission, security requirements, policies, and compliance requirements). Owned and managed by one or more organizations, third parties, or a combination of the two. Can be on premise or off premise.

Public Cloud

Cloud infrastructure provided to the public. Owned and managed by a single business, academic, government organization, or a combination. On premise for cloud providers, off premise for users.

Chapter 3

Migration to cloud

Before to decide we need to understand if the cloud is good for our business. The analysis of the characteristics and advantages of the cloud with respect to on-premise systems can tell whether the cloud is good for our business.

The aspects to consider are mostly three:

- Strategic: most of the time is build on money
- Financial: we should improve the economy of our business reducing the cost and increase the financial benefit
- Customer

Know where the drawbacks are and additional business functionalities are needed. Identify the challenges to be faces. Define the means to address the challenges.

We need to asses performance, understand performance before and after migration. We have an idea that a movement to the cloud could have an impact to our business.

Key questions:

- When and how we should migrate an application to the cloud?
- Which part or component of an IT application can/must be migrated to the cloud and which ones not?
- Which kind of customers will benefit from IT migration to the cloud? Which benefits for business?

Why migrate:

- On-demand resourcing
- Scalability
- Economy of scale
- Flexibility and elasticity
- Growth
- Utility based metering
- Shared infrastructure
- High availability
- Security

3.1 On-demand resourcing

In on premise service if we need to adding additional resources, computation, storage, network, requires a long purchasing process:

- Contacting the supplier
- Obtaining a price
- Ordering the hardware
- Installing, configuring, cabling the hardware in a data center

Process requiring weeks, even though days/hours can be too much in some cases. Risk of losing customers.

With the cloud an allocation process almost is immediate. Can allocate where and when is needed:

- If I have a CPU usage spike I can power on a new server in seconds

On-premises issues are solved with an almost immediate access to resources that are being selected and configuring by choosing through a series of options.

3.2 Scalability

In on premise service is not supported. Scalability in this systems requires significant financial resources and space in the data center.

In cloud scalability offers scaling up and scaling out resources depending on requirements and requests of services and applications:

- Scaling up and scaling down allow modifying the power of an instance
- Scaling in and scaling out add or remove the number of instances

Scalability is possible because the cloud supports the concept of on-demand resourcing.

3.3 Economy of scale

On cloud resource sharing among tenants and the huge amount of resources provided by a public cloud allow offering computing, storage and network at very low prices. The more you buy, the cheaper it becomes. Cloud resources are much more economical than the same ones on on-premise infrastructure.

In on premise traditional hosting costs are much higher for unit of resource.

3.4 Flexibility and elasticity

Cloud computing offers huge flexibility and elasticity. It's possible to choose how many resources without establishing the needed power in advance. The infrastructure can adapt, like an accordion.

On premise is difficult to manage spikes. Resources must be planned in advance to be able to handle spikes. If planning is wrong then problems with customers and loss of reputation. (underestimate or overestimate resources).

3.5 Growth

On cloud is almost immediate support for every growth profile. Limits to the grow are reduced compared to classical environments.

In traditional systems the business growth could require:

- Buying a new office
- Hiring new workers
- Waiting for months

3.6 Utility based metering

In cloud it's pay per use, we pay for what we use and nothing more, also we can turning off servers when they are not needed.

In on-premise servers are kept on 24/7/365. Electricity and cooling costs, wear out..

3.7 Shared Infrastructure

In the cloud hosts are virtualized. Different tenants share the same resources.

On premise we have no sharing.

3.7.1 High availability

In cloud native support for replicating resources and services. Replicas among several zones and geographical regions. Important to understand which part of the resilience depend on the vendor and which part on the user.

In other way supporting on-premises high availability requires higher costs and competence often not affordable to medium sized companies. Often medium sized companies do not have replicated sites and do not have advanced disaster recovery programs.

3.8 Security

Cloud provides infrastructures often already certificated and compliant to standard. For example: PCI DSS, ISO, HIPAA, SOX.

In on premise is mostly security. Almost no assurance, compliance, certification.

3.9 Migration

We have to evaluate the costs. The important is the Total Cost of Ownerships will decrease or encrease and the retourn of investment.

We have 5 levels of migration:

- application
- code
- design
- architecture
- usage

Nowadays we are building cloud native application.

Migration levels are applied to the different IaaS, PaaS, SaaS levels.

The application once moved to the cloud could be splitted in:

- Application after cloud migration
- Part of the application executed locally
- Part of the application optimized for the cloud

$$P \hookrightarrow$$

3.10 Migration 7-step model

- Cloud Migration Assessment
- Isolate the dependencies
- Map the messaging & the environment
- Re-architect and implement the lost functionalities
- Leverage cloud functionalities & features
- Test the migration
- Iterate and optimize

3.10.1 Cloud migration Assessment

- Evaluation of the problems at the 5 levels of migration
- Consider tools to use, test cases, functionality, configurations, non-functional requirements of enterprises
- Analyze prototypes and proof-of-concepts of the technologies to be evaluated (like going to shop trying a car)
- Produce a migration strategy

3.10.2 Isolated dependencies

Which are the dependencies mandatory to run our system.

- Isolate system and environmental dependencies of application components in the local data center
- Model the level of complexity of the migration

3.10.3 Map the messaging and the environment

Map what needs to stay local and what needs to be migrated.

3.10.4 Re-Architect and implement the lost functionalities

Identify what needs to be redesigned and re-implemented. In some cases, some functionality may be lost.

3.10.5 Leverage cloud functionalities and features

Use cloud capabilities to implement the service and possibly improve it. The idea is not just improve the infrastructure but improve the business 360.

3.10.6 Test the migration

- Test the correct functioning of the service on the cloud
- Identify migration risk

3.10.7 Iterate and optimize

- If the test yields conflicting results, iterate the process to optimize the application migration to the cloud
- Manage the migration risks identified in step 6

3.11 More into detail

Migration must consider:

- Time management
- Security
- Choosing vendors
- Choosing a cloud model
- Assessing service readiness for migration

- Aligning services
- Design
- Migrating deployment options
- Optimization
- Business continuity

3.11.1 Time Management

Moving to the cloud is not free, we'll spend several times. It's a measure for success, key milestones. Phased approach, communication with the team. Security compliance. Testing. No shortcuts, follow the phases.

3.11.2 Security

We are moving from private to public. The idea is to manage assets and encrypt data and we need to share responsibility to the server provider.

3.11.3 Vendor selection

When you migrate to the cloud we have to decide your vendor. It depends on your scenario.

- Major generic vendors, some specific vendors
 - What to migrate? Entire infrastructure? Example AWS
 - Only storage? Example Dropbox
- Required security must be ensured.
- Range of service:
 - Migration services
 - Development services
- Features (e.g., data persistence guarantee, supported operating systems)
- Offered interfaces: API, GUI, etc.
- Multi-vendor Configuration?
 - Best fit vs integration issues
- Pricing structure
- Support SLAs, etc.

3.11.4 Deployment model

Public - private - hybrid - community Key point is to choose the deployment model based to the data sensitivity that we are moving to the cloud.

3.12 Service readiness

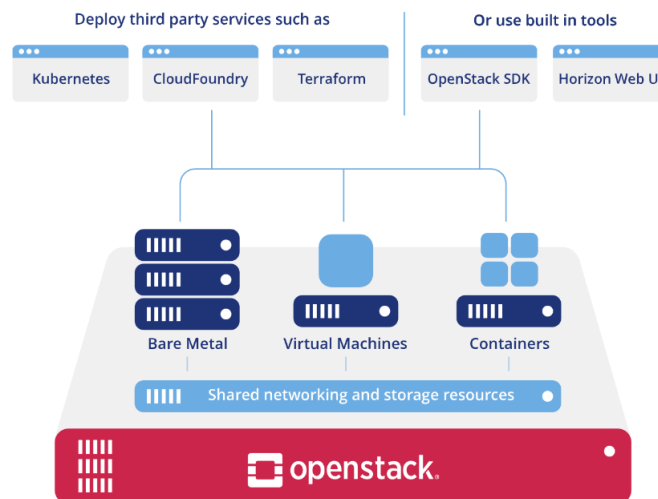
- De-coupled structure
 - Different parts that scale independently
- Performance
 - Test environment
- Compatibility
- External dependencies

Chapter 4

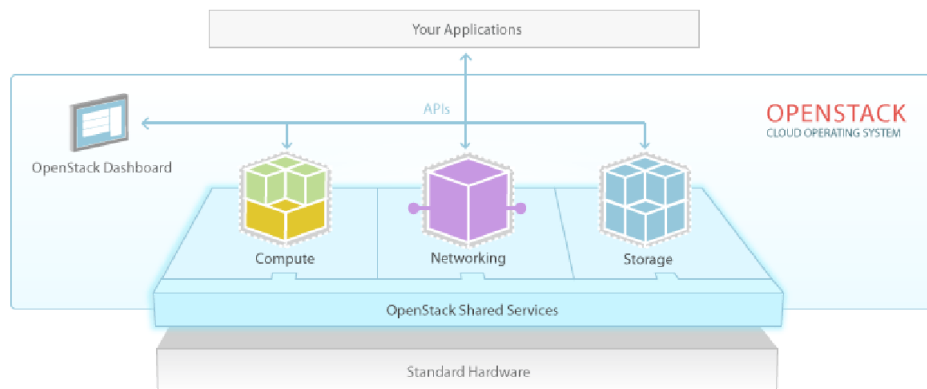
IaaS

4.1 OpenStack

Openstack is a cloud operating system that controls large pools of compute, storage, and networking resources. All resources are managed and provisioned through APIs or a dashboard. A dashboard is also available. Beyond standard infrastructure-as-a-service functionality, additional components provide orchestration, fault management and service management amongst other services to ensure high availability of user applications.



Open source software for private and public cloud creation. Cloud operating system that controls a pool of resources: compute, storage, networking.



Provide users with a trustworthy and configurable cloud solution. Simple to implement, provide scalability and extensibility, provide many advanced functionalities. Inter-connected services implement different components of the cloud infrastructure. Can be accessed through APIs or dashboard.

4.1.1 Dashboard

We can manage from the dashboard the complete infrastructure. Implement a GUI to access, retrieve, and automatize cloud resources. Permit to deliver third-party services like billing and monitoring Extensible web application to control compute, storage and

networking resources. Provide an overview on the cloud dimension and status. Permit to create users and projects, assign users to projects and limit the use of resources.

4.1.2 Compute

OpenStack supports providers in the release of computing resources on demand. Resources accessed through APIs or web interface. Developed for horizontal scaling. This is the part that allow us to do some automation.

Provide a flexible architecture (Open source, Can be integrated with third parties and legacy systems) Manage and automate resource distribution and support virtualization techniques (Xen, KVM).

Functionalities:

- Manage virtualized resources
- Support for LAN (DHCP, IPv6)
- Authentication and API usage
- Synchronous and distributed architecture
- VM image management (also live)
- Floating IP
- Security group
- RBAC
- Projects and quota

4.1.3 Storage

It's the third pillar of each infrastructure in the cloud. Support both Object Storage and Block Storage Object Storage provides storage with optimized costs and ability to scale out:

- Storage platform fully distributed, can be accessed through APIs, can be integrated with applications or used for storage
- It is not a traditional file system, it stores data in objects
- OpenStack manages replica and supports horizontal scalability

Block Storage permits to connect block devices to compute instances to achieve greater performance and integrates with enterprise storage platforms:

- Manage creation, mounting and unmounting of block storage
- File divided in fixed data blocks
- Integrated with OpenStack compute and dashboard

Functionality:

- Based on common hardware
- Scalability support
- Unlimited storage
- Replica support
- No central DB
- Can be integrated with compute
- Support S3 APIs
- Snapshot and backup per block volume

4.1.4 Network

The idea here is to build our virtual network and open it to the world. Datacenter network contains an ever-increasing number of servers, network equipments, storage systems, and security appliances:

- These devices are divided in VMs and virtual networks
- IP address, routing configurations, security rules grew exponentially
- Traditional approaches to network management do not provide automatic and scalable support
- Users require more control and flexibility

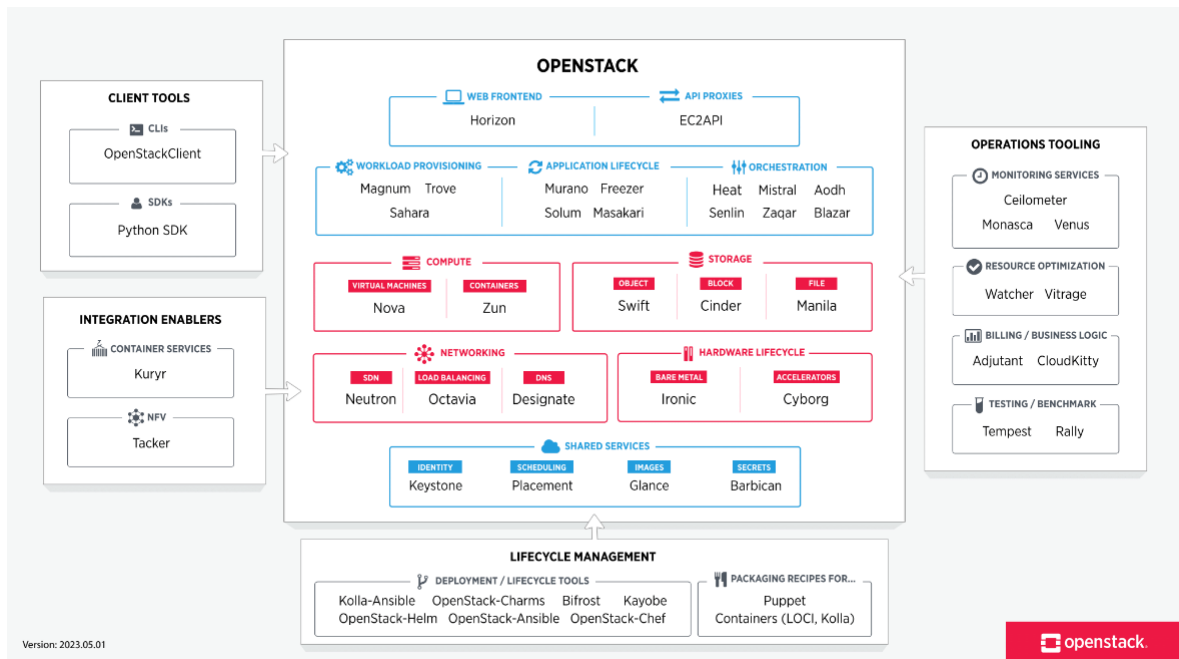
OpenStack Networking is scalable, based on APIs and “pluggable”:

- Manage network and IP address
- Ensure no bottleneck or limitations in a cloud deployment
- Users create their own network, control traffic and connect servers and devices

4.1.5 Services

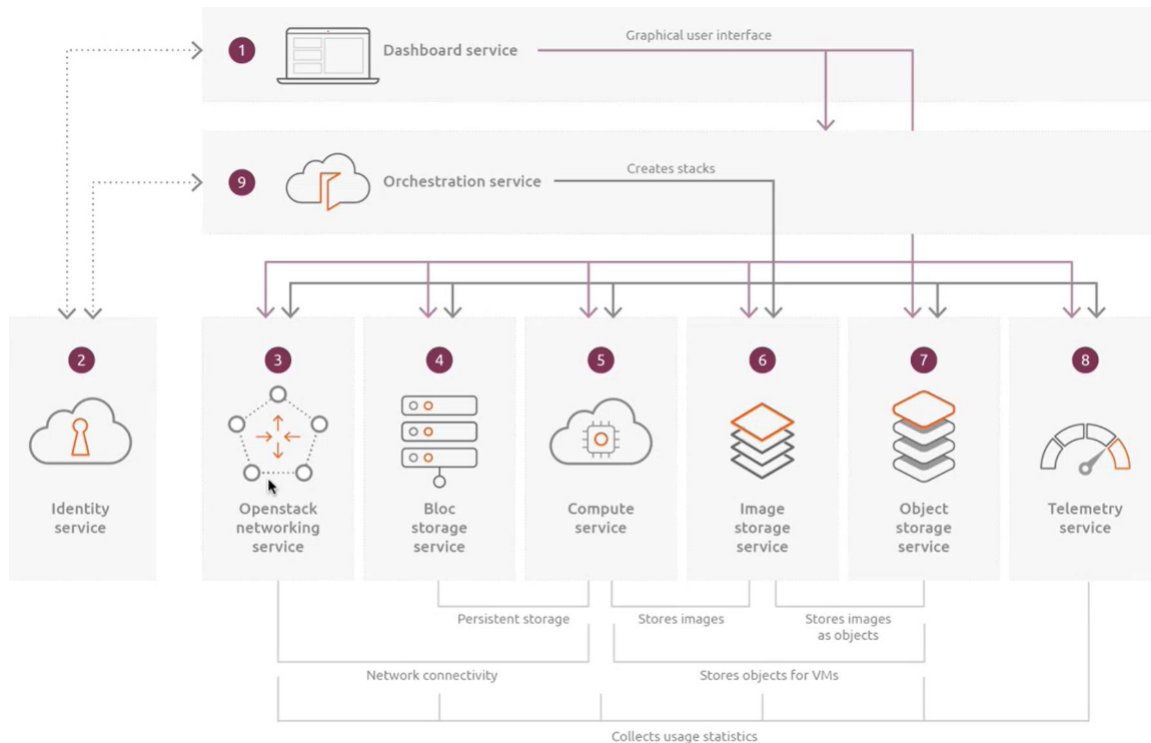
As we said OpenStack is build by a variety of services:

- Identity Service
- Image Service: managing the virtual machine images
- Telemetry Service: monitoring what is happening on the infrastructure
- Orchestration Service: the idea of put together services
- Database Service



We can see that OpenStack is not a , but is modular, he has several different modules and each of these module evolve independently because the API abstract the evolution of internal to the external.

In this case we can see better the flow of the components:



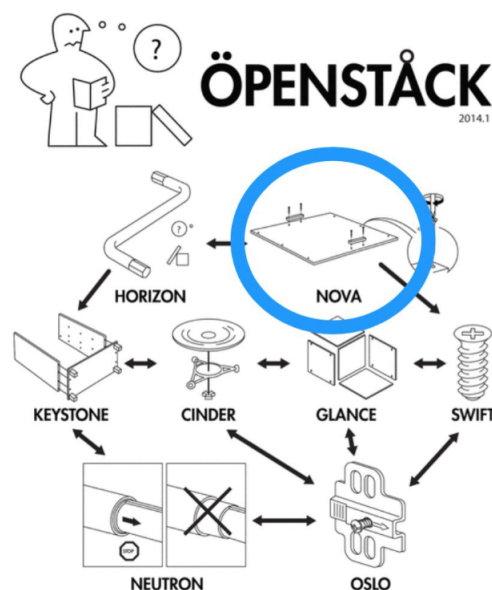
4.1.6 Mapping service/project

- Dashboard -i Horizon
- Compute -i Nova
- Networking -i Neutron
- Object storage -i Swift
- Block storage -i Cinder
- Identity -i Keystone
- Image -i Glance
- Telemetry -i Ceilometer
- Orchestration -i Heat
- Database -i Trove

4.1.7 Horizon

Modular web application based on Django (web framework based on python language). Dashboard can be accessed by clients and APIs of any OpenStack services. Admin endpoints provide admin functionalities through the dashboard. Permit to execute all operations for the management of the infrastructure and account. A few clicks to deploy/un-deploy a VM, create volumes, manage security.

Compute - Nova

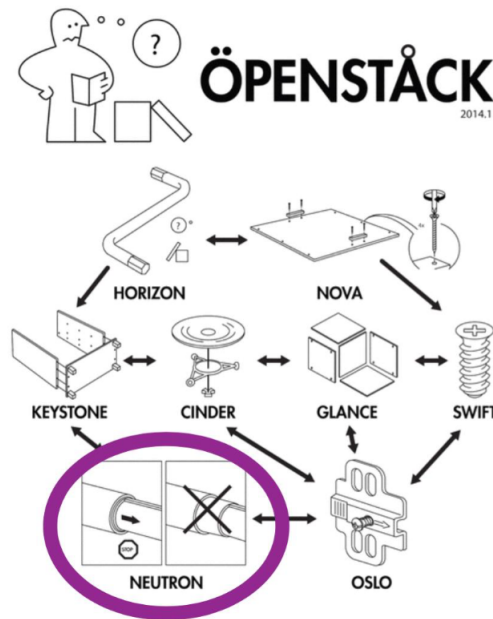


Compute

- KVM
- VMWare
- Hyper-V
- XenServer
- PowerKVM

This is the Fundamental service of OpenStack infrastructure. Manage the entire life-cycle of the VM instances in the virtualized environment (Creation, coordination, deletion of VMs). Interact with the identity service for authentication, with the image service to access disk and server images, and dashboard to provide admin and user interface. Horizontal scalability on standard hardware and retrieve images needed for instance creation. Nova-network demon used for network management.

Neutron



Network

- OpenvSwitch (ML2)
- VLAN
- VxLAN
- Flat Network
- Provider Networks
- 3rd parties
- NSX
- OpenContrail/Contrail
- & others

Neutron service is the networking service (neutron-server). Permit to create and implement in the virtual network devices and interface managed by other OpenStack services. Plug-in: full flexibility, permit to manage and interconnect different network devices and software. Interact with compute service to provide a network infrastructure and support network access to the instances.

Storage

The idea is the storage is virtualized not real, ephemeral means that we can delete and add storage as we want.

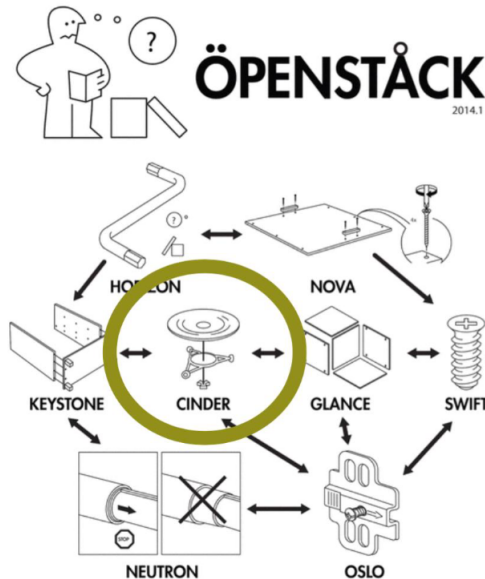
On-instance / ephemeral:

- Provide space from zero
- Associated with VM
- Can be accessed when the VM is active
- Support flavor-based restrictions

And as we said before we have two type of storage:

- Block Storage
- Object Storage

Cinder is the service for the block storage:



Storage

- Software Defined
 - CEPH
 - Gluster
 - HPE LeftHand
 - IBM GPFS
- Traditional
 - NetApp
 - HPE 3PAR & XP
 - Dell StorageCenter
 - EMC VMAX
 - Hitachi
 - Nexenta
 - ... many others

Full list:
<https://github.com/openstack/cinder/>

- Add persistent storage to a VM
- Data maintained until the VM is canceled
- Provide an infrastructure to manage volumes and volume snapshots
- Interact with OpenStack Compute
- Can be accessed by VMs
- Mounted through OpenStack Block Storage controlled protocol (e.g., iSCSI)
- Storage dimension depends on the needs

Object Storage - Swift

Multi-tenant object storage system(Used to store data and images of VMs). Data maintained until the VM is canceled. Can be accessed everywhere. Manage big amount of data and can scale using RESTful HTTP API. Provide abstractions to store methods, non storage itself. Can be executed independently from Compute Nova.

This is independent from the computation, this is the solution to store data (object) independent from the virtual machine.

Identity - Keystone

Two main functions:

- Track users and corresponding privileges
- Provide a list of services available with API endpoint

At service installation time, for identity management, any services in the OpenStack infrastructure is registered, At this point, the identity service knows active services and where they are located in the network.

Keystone generates authorization tokens for users:

- Using keystone APIs, a user submits its credentials and retrieves the authentication token
- Maintain a table with users and privileges

This is a visual example of an interaction through keystone:

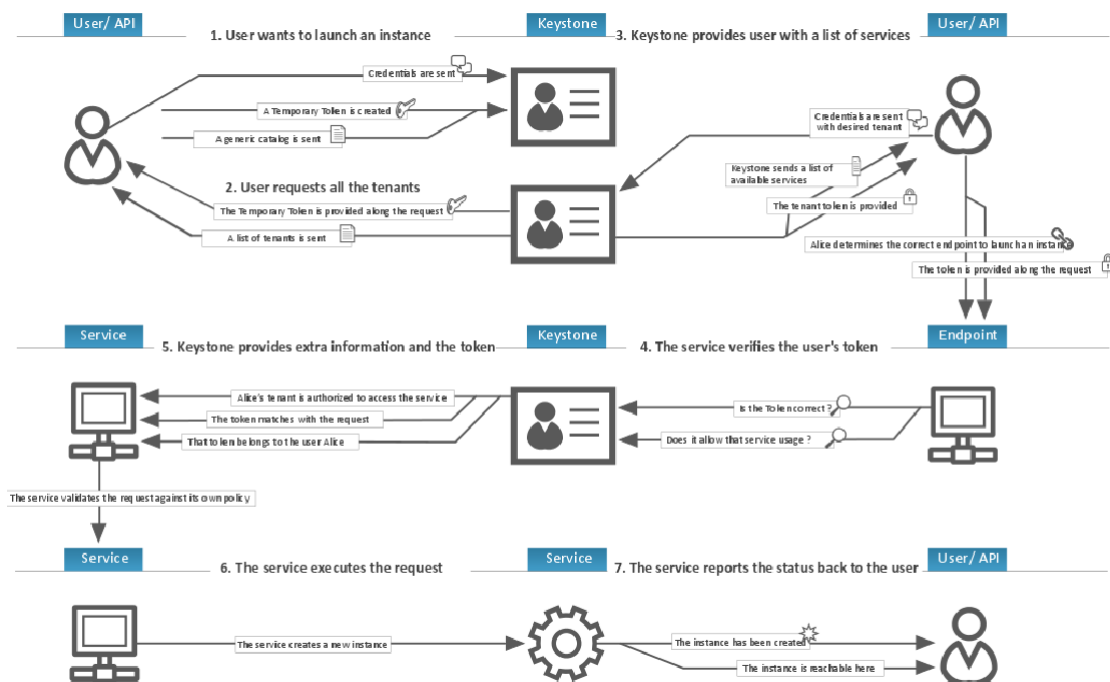


Image – Glance

Important for IaaS, it manages images, basically is a images repository and is used by Nova that take an image and start the Vm. Accept requests for disk images or server images, VMs, or metadata corresponding to images of end users or OpenStack Compute components. Support storage of disk images and server images on different types of repositories including OpenStack Object Storage. Support caching, replication to provide consistency and availability, auditing.

Telemetry - Ceilometer

Collect metering data from OpenStack services (CPU, network usage). Useful to calculate the bill (pay-as-you-go). Collect events and metering data monitoring notifications sent by services. Publish data through data store and message queue. Raise alarms when collected data violates predefined rules. Contain: compute-agent, central-agent, notification-agent, collector, alarm evaluator, alarm notifier, API server.

It's the module that provide us function to handle specific behaviour of the infrastructure.

Orchestration – Heat

Provide a template-based where we orchestrate different solution and to describe the infrastructure deploy, orchestration

- Permit to describe and automatize the infrastructure deployment
- Execution of calls to OpenStack API to generate executable cloud applications

The template language permits to:

- Specify configurations of compute, storage and network
- Specify post-deployment activities to automate the provisioning of infrastructure, services, and applications
- Create many types of OpenStack resources: instances, floating IPs, volumes, security groups and users

Heat permits developers to directly integrate the orchestration module of by means of custom plug-in, provide high availability and nested stack.

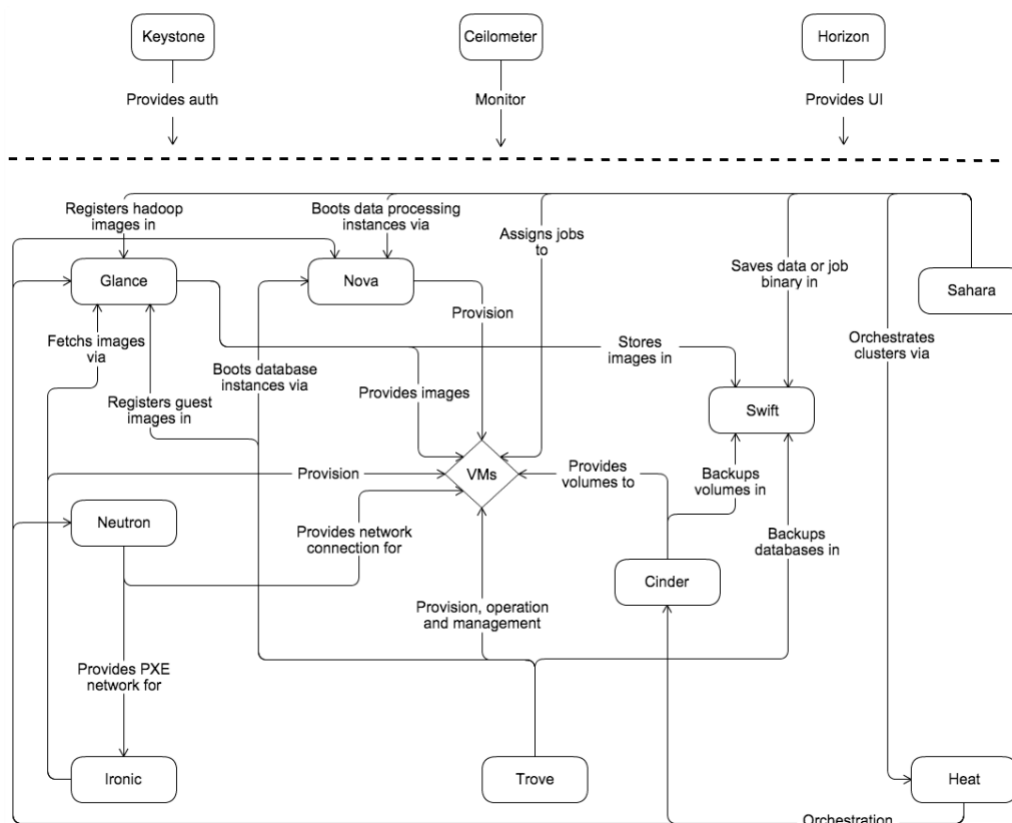
4.1.8 Network

We have 4 network:

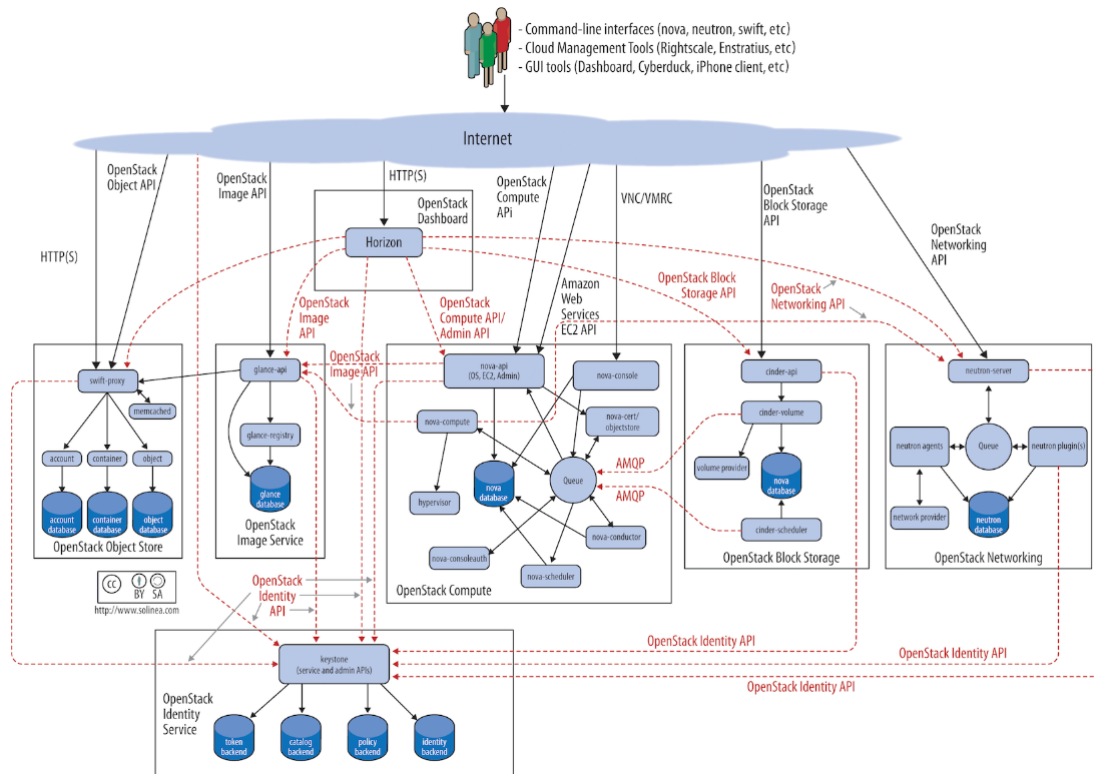
- Management network:
 - Dedicated to internal communications between processes
 - The IP addresses on this network should be reachable only within the data center
 - It is the Management Security Domain
- Data network:
 - Used for VM data communication within the cloud deployment
 - The IP addressing depends on the network requirements
 - It is the Guest Security Domain
- External network:

- Used to provide VMs with Internet access in some deployment scenarios
- The IP addresses on this network should be reachable by anyone on the Internet
- It is the Public Security Domain
- API network:
 - Exposes all OpenStack APIs, including the OpenStack Networking API, to tenants
 - The IP addresses on this network should be reachable by anyone on the Internet.
 - It is the Public Security Domain

4.1.9 Conceptual Architecture



4.1.10 Logical Architecture



As we can see all the building block pass through the identity service before executing all the activity.

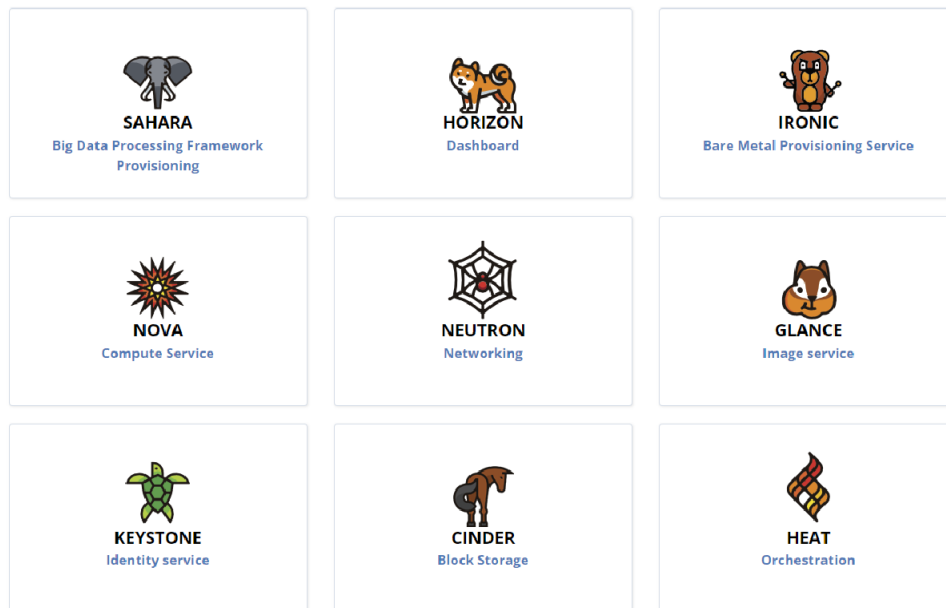
All services authenticate using the Identity service. Services interact through public APIs with the unique exception for privileged admin commands. OpenStack services composed by different processes, all services have an API process waiting for requests, pre-processing requests and distributing them to the entity responsible. Communications between processes happens with an Advanced Message Queuing Protocol (AMQP) message broker; service status stored in a database we have different options: RabbitMQ, MySQL, MariaDB, and SQLite. Access to OpenStack through a web interface (implemented from dashboard service), client from command line, and API request with browser plug-in or curl.

4.1.11 Use case: Big Data

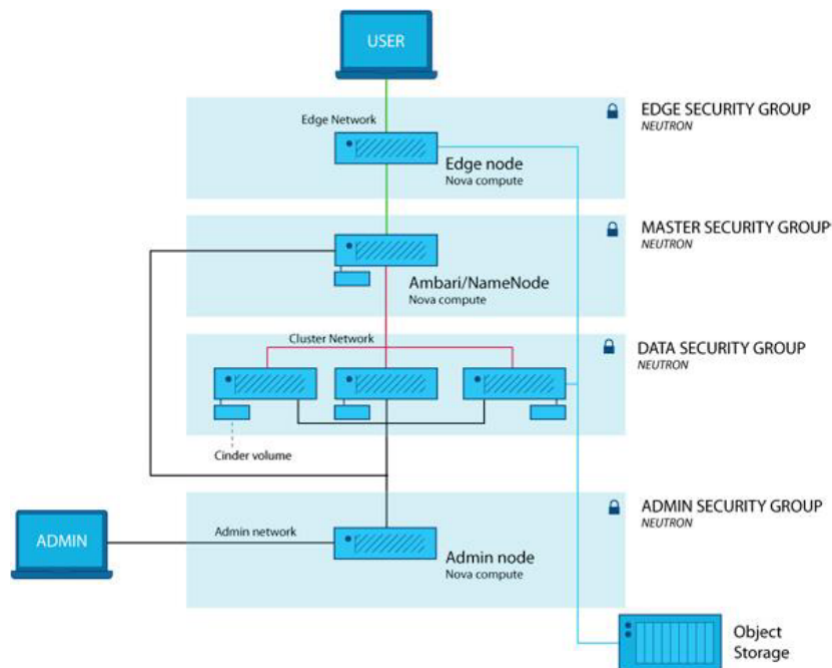
Having a complete infrastructure that is able to collect, maintain and analyze data is a great thing.

Providing scalable, elastic infrastructure for big data collection and analytics this is the primary use case for OpenStack. An OpenStack-based cloud environment offers automated Big Data cluster provisioning to service high volumes of analytics requests with rapid deployment time requirements. Big Data reference architecture to learn how to use OpenStack services to build and automatically scale an Apache Hadoop Big Data cluster on an existing OpenStack cloud.

With open stack we can use different project to provide big data services:



In this image we can see an architecture for big data, it's a possible deployment for big data use case:



4.2 Amazon AWS

4.2.1 Amazon EC2

Amazon Web Services offers digital infrastructure services since 2006. AWS Cloud is available in more than 30 geographic regions across the globe and provides resizable

compute capacity in the cloud. It makes web-scale computing easier for developers and provides a simple web service interface. It provides simple APIs and many services to support web-based computing. Pay-per-use.

4.2.2 Global Infrastructure

Amazon EC2 is hosted in multiple locations around the world. It is composed of the following building blocks:

- Regions, separate geographic areas
- Availability Zones, multiple and isolated locations within each Region
- Local Zones, support the placement of resources, e.g., compute and storage, in locations closer to end users
- AWS Outposts, move native AWS services, infrastructure, and operating models on-premises
- Wavelength Zones, support developers in building applications that provide ultra-low latencies to 5G devices and end users:
 - Wavelength deploys standard AWS compute and storage services to the edge of telecommunication carriers' 5G networks

Each Region is isolated from others:

- Support high fault tolerance and stability
- Only resources that are tied to the Region can be seen/accessed

Endpoints are used to programmatically connect to AWS services:

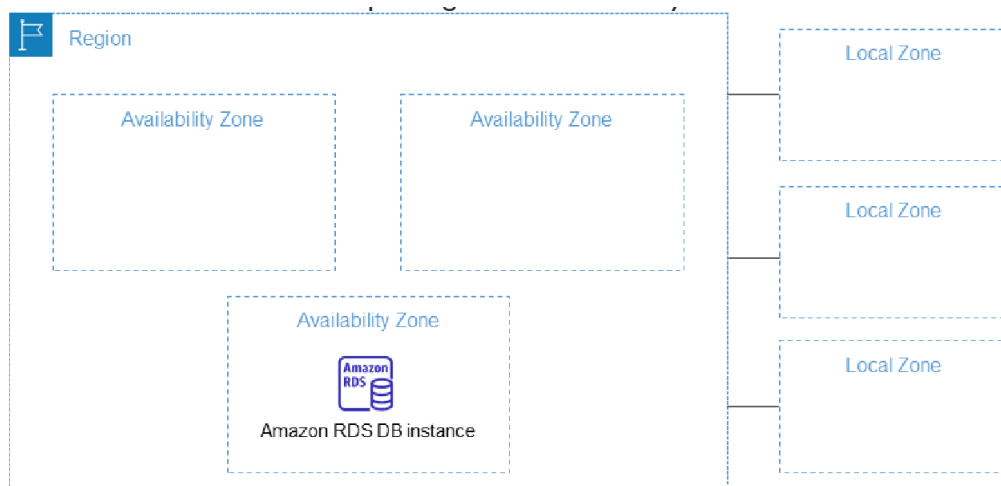
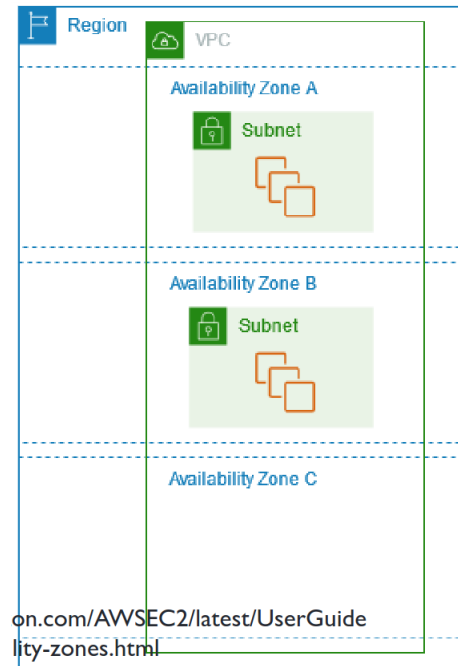
Region Name	Region	Endpoint	Protocol
US East (Ohio)	us-east-2	ec2.us-east-2.amazonaws.com	HTTP and HTTPS
		ec2-fips.us-east-2.amazonaws.com	HTTPS
		ec2.us-east-2.api.aws	HTTPS

Each Region has multiple, isolated locations known as Availability Zones:

- Code for Availability Zone is its Region code followed by a letter identifier (e.g., us-east-1a)

To launch an instance, the user selects a Region and a virtual private cloud (VPC), and possibly a subnet from one of the Availability Zones:

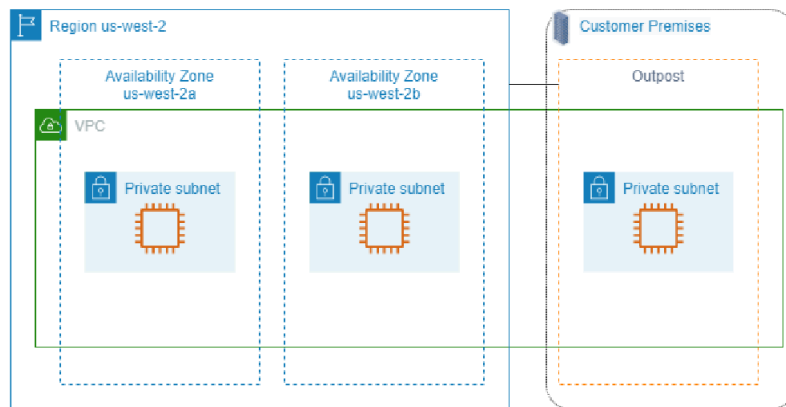
- Availability Zones can make instances resilient to failures



- If one instance in an availability zone fails, you can move the traffic to another instance in another availability zone

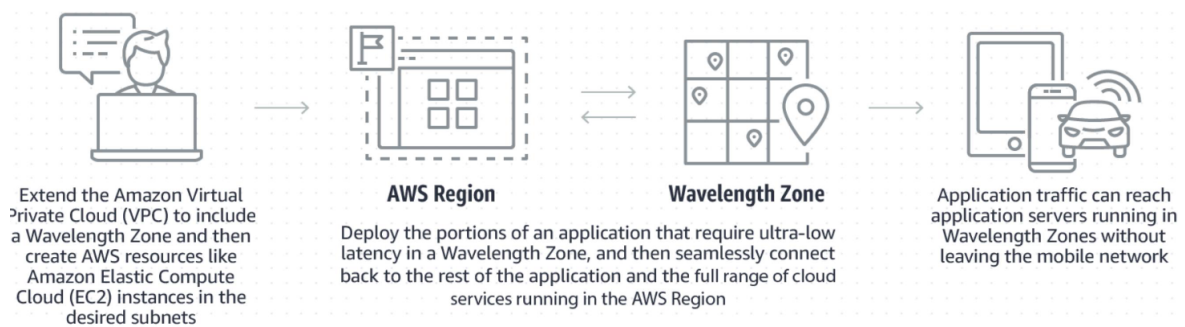
For example, Amazon RDS enables you to place resources, such as DB instances, and data in multiple locations:

AWS Outposts extends AWS infrastructure, services, APIs, and tools to customer premises. Customers build and run applications on premises using the same programming interfaces as in AWS Regions, while using local compute and storage resources. An Outpost is a pool of AWS compute and storage deployed at a customer site. Communications with other instances in the AWS Region happen using private IP addresses (VPC).

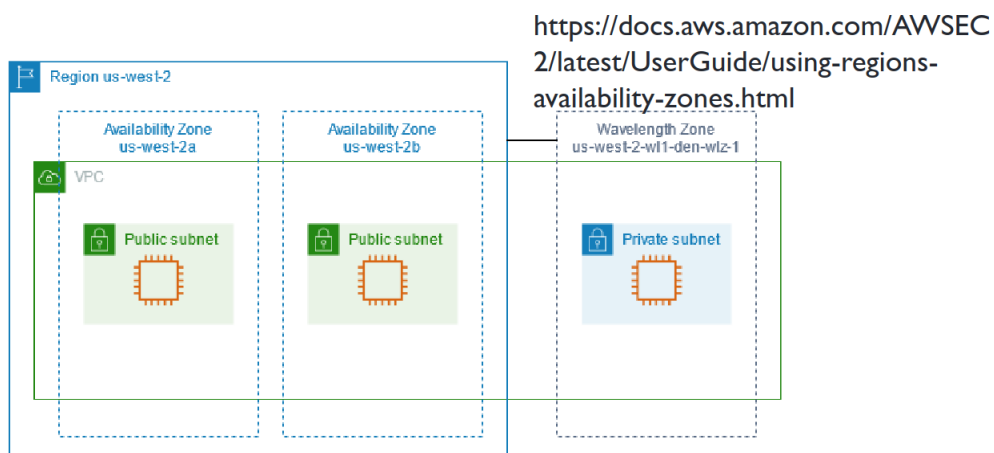


4.2.3 Wavelengths

Wavelength Zones, support developers in building applications that provide ultra-low latencies to 5G devices and end users, Wavelength deploys standard AWS compute and storage services to the edge of telecommunication carriers' 5G networks.

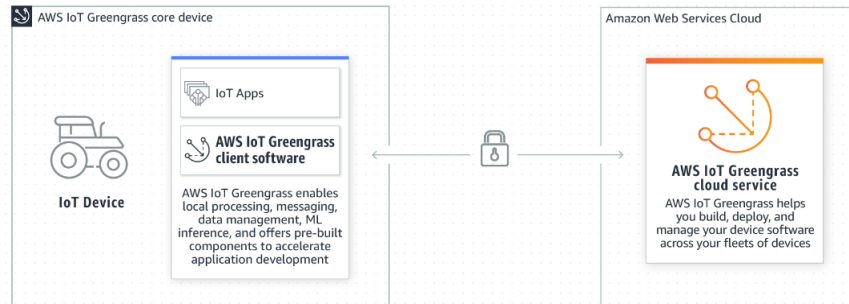


Virtual private cloud (VPC) can be extended to one or more Wavelength Zones. A Wavelength Zone is isolated in the carrier location where the Wavelength infrastructure is deployed and tied to a Region. Wavelength Zone code: its Region code followed by an identifier that indicates the physical location (e.g., us-east-1-wl1-bos-wlz-1 in Boston).



4.2.4 Greengrass

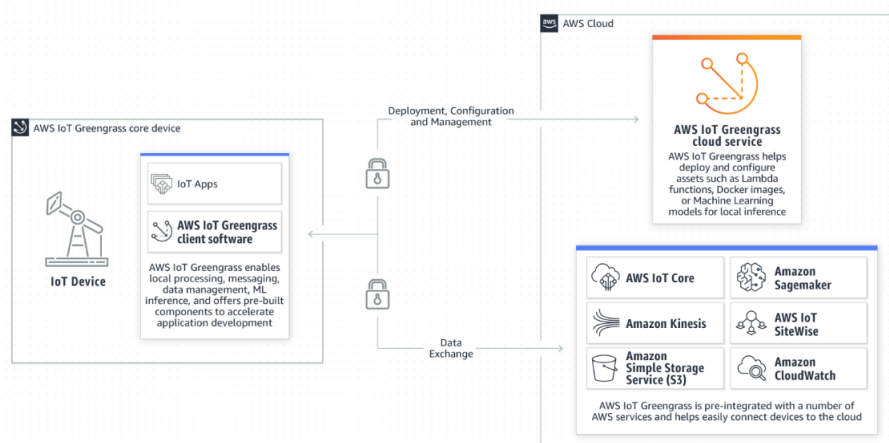
AWS IoT Greengrass is an open-source edge runtime and cloud service for building, deploying, and managing device software.



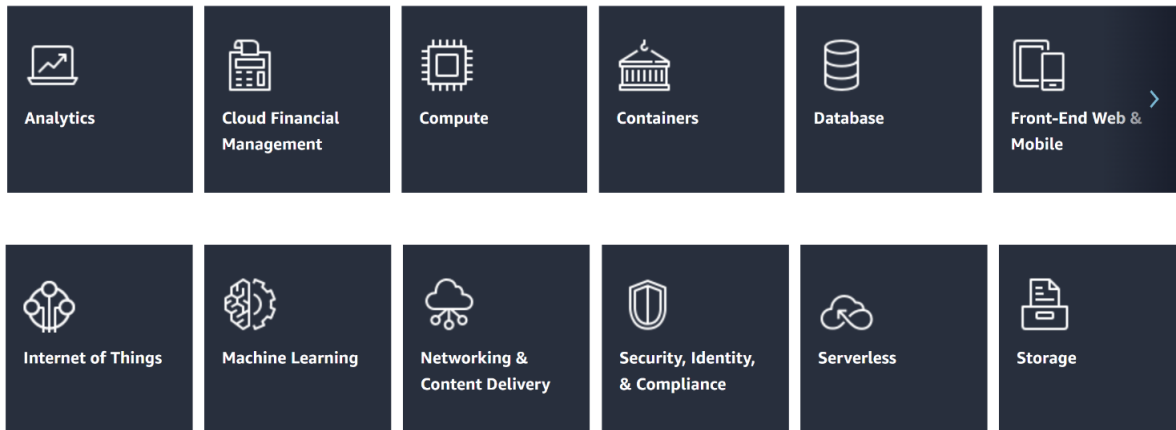
AWS IoT Greengrass permits to connect a variety of heterogeneous devices from minuscule sensors to large appliances.



AWS IoT Greengrass permits to quickly connect to AWS services or third-party applications at the edge.



Amazon AWS is Much More:

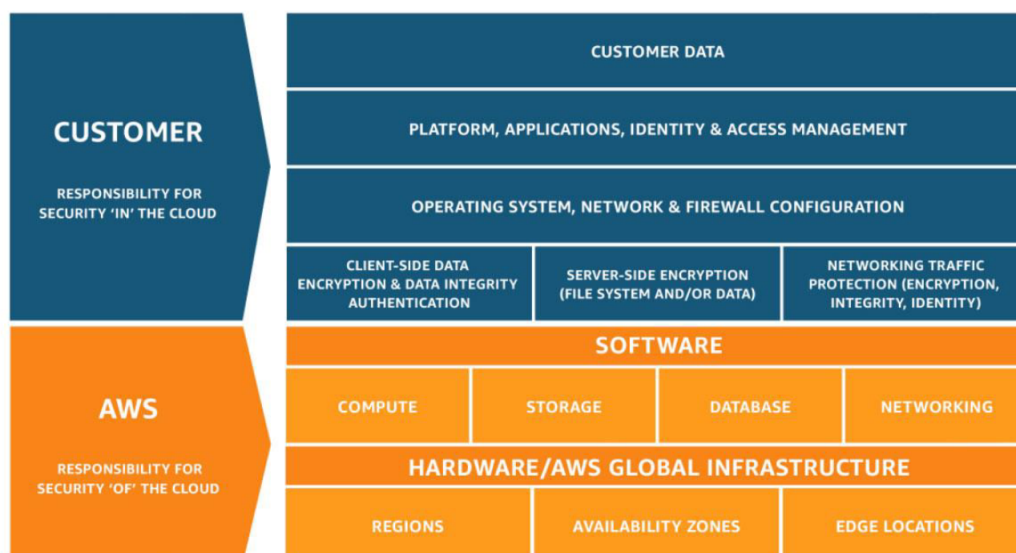


4.2.5 AWS GovCloud

It's Amazon Region designed and developed that supports security and compliance requirements of US Government. Comply with the FedRAMP High baseline; the DOJ's Criminal Justice Information Systems (CJIS) Security Policy; U.S. International Traffic in Arms Regulations (ITAR); Export Administration Regulations (EAR); Department of Defense (DoD) Cloud Computing Security Requirements Guide (SRG) for Impact Levels 2, 4 and 5; FIPS 140-2; IRS-1075... AWS GovCloud (US-East) and (US-West) Regions are only available to U.S. entities and root account holders who pass a screening process.

4.2.6 Shared Responsibility

Means that when there is a fault we have to understand who is responsible. We don't have a single person that is responsible for all the security, but we have several person that are responsible for part of security. The security depends on multiple entities.

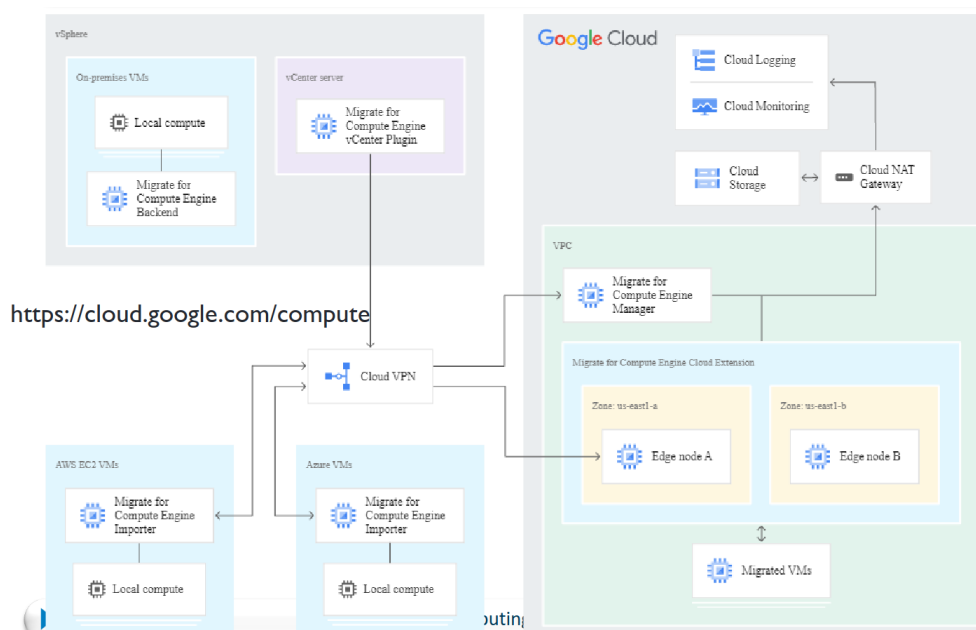
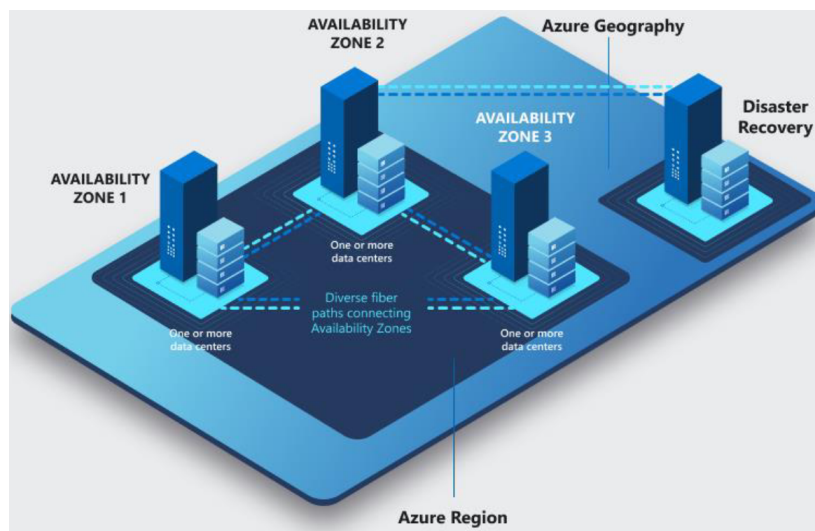


4.3 Other IaaS Providers

We have many other providers:

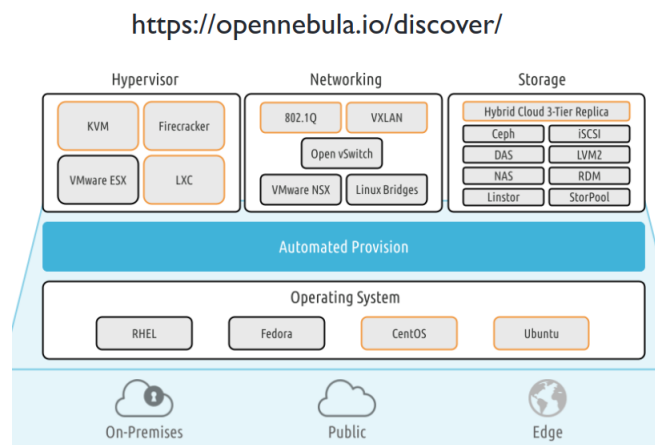
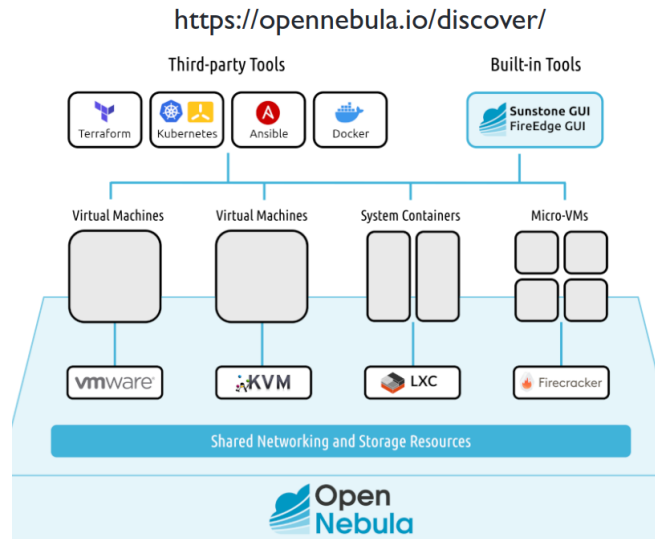
- Azure
- Google Compute Engine

The idea that we have to understand that they are very similar. We don't know how they are implemented but we know that they are divided in zones and how the zones are connected. On other important aspect is the latency, because again something is available if the latency is low.



4.3.1 Open Nebula

The idea is to deploy hybrid and edge environments connecting resources from AWS, Google and Equinix:



Chapter 5

Recap Distributed System

While we distribute a system we'll have some issues, one of that is observability this is why the system appears as a single coherent system but it's not.

There are many distributed system across 3 dimensional space.

The network is very important, is the way the nodes can communicate.

Why the distribution is important? can provide some properties:

- Separation of functionalities in order to provide them in an efficient way, like Netflix
- Inherent distribution: distributed by default, like in domotic in which we have sensors. Or when people are working in collaborative way and they share data in distributed system
- Reliability
- Economy

Middleware is the notion that is equivalent to the operating system for distributed system. It's made of protocols, channels we can have several design for different distributed system.

- Openness: interoperable, it has to be open to other application that want to plug in our system
- Scalability: not just in term of local scalability (go bigger) but also in geographical scalability (decide the location of nodes). Also administrative scalability, adding nodes without issues.
 - Scale out: increasing the capability of network or adding nodes. Load balance on top, could be a problem for transaction, replica of data
 - Scale up: increasing the capability of the single virtual private server, improving number of CPU...

The problem is to provide transparency. We have different types of transparency:

Transparency	Description
Access	Hide differences in data representation and how an object is accessed
Location	Hide where an object is located
Relocation	Hide that an object may be moved to another location while in use
Migration	Hide that an object may move to another location
Replication	Hide that an object is replicated
Concurrency	Hide that an object may be shared by several independent users
Failure	Hide the failure and recovery of an object

Transparency migration is the migration of services not servers. Fault transparency is what they'll ask to us during the exam, three nodes application, what happens if we turn off one node?

Distribution transparency not only comes at a performance price, in practical situations it can never be fully achieved.

Be careful about wrong assumptions:

- network latency is not significant. Could be achieved on the local host ONLY. Local deployment and test is fine, but is like having the assumption of zero latency.
- network is reliable, static, secure, and homogeneous

Parallel computation is not necessarily a distributed system, is more HW.

5.1 Architectures

We can make distinction in:

- software architecture: logical organization of the software (components interaction, structure, level of independency etc.)
- system architecture: where the components that constitute a distributed system are placed across the various machines.

We can have several combination of these:

- Layered
 - Components organized in layers
 - Interface, Service and Protocols
 - Application layering: interface, processing, data
- Object-based: Object distributed in different machines
 - Encapsulation
 - Separation between interface shared between machines and object present just in one (or some) machine
- Resource-based
 - REST: Distributed system as a collection of resources individually managed by components
- Publish-subscribe
 - Loosely couple dependencies between coordination and processing
 - Different degree of temporal and reverential coupling
 - A way to be synchronous in a distributed system, supporting order

5.2 Design patterns

- Middleware design patterns:
 - Wrappers/adapters and interceptors (openness goal)
- Wrappers: transform legacy system
 - Broker or 1 to 1
- Interceptors: break the communication flow allowing other code to be executed
- System architecture: instance of a Software architecture

5.3 ZeroMQ

It is a high-performance asynchronous messaging library, aimed at use in distributed or concurrent applications. Unlike message-oriented middleware, a ZeroMQ system can run without a dedicated message broker. It supports several patterns as: pub/sub, request/reply, client/server etc.

5.4 Kafka

Kafka is a distributed streaming platform:

- High Scalable (partition)
- Fault tolerant (replication)
- Allow high level of parallelism and decoupling between data producers and data consumers. Producer-consumer based paradigm.

It's de facto standard for near real-time store, access and process data streams.

In Kafka, a Topic is a user-defined category to which messages are published. Kafka Producers publish messages to one or more topics. Consumers subscribe to topics and process the published messages. Cluster consists of one or more servers, called Brokers that manage the persistence and replication of message data.

Broker: Kafka node on the cluster

Topics: Stream of records category

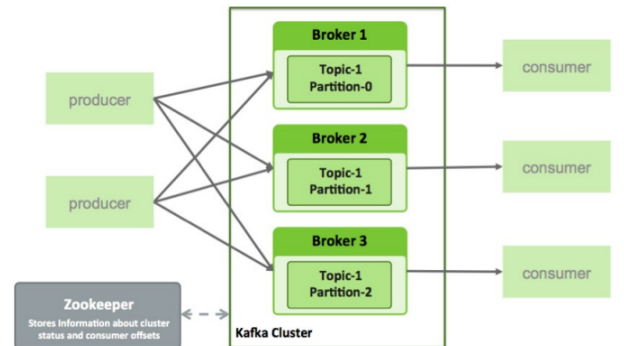
- Multiple writers and readers
- Partitioned
- Replicated

Consumer: pulls messages off of a Kafka topic

Producer: push messages into a Kafka topic

Data Retention:

- Based on time or size



Zookeeper: Stores Kafka Metadata

5.5 Mino Object - Distributed file system

It's a portable high-performance object storage system designed for kubernetes. It seems that you have a file system which is local and can use it like a local one, but it's not local part of file system is away. We have something similar when we mount one drive somewhere, it seems to be local. From the application point of view it has to wait a little bit.

5.6 Process

Process and threads are important even if we are not doing parallel computing. The threads are useful to not stop process in case of blocking I/O. We also have the concept of virtualization of process and in general resources.

Chapter 6

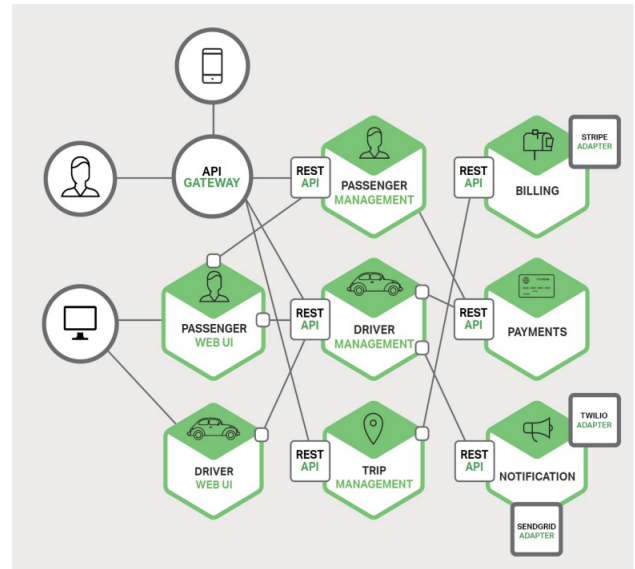
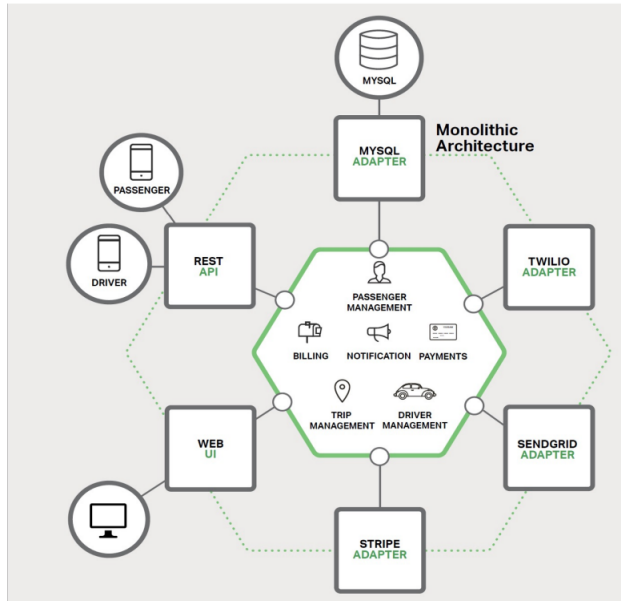
Containers and Docker

6.1 Microservices Containerization

Peculiarities:

- Functional decomposition: vertical cuts not on layers. Important for software point of view, divide architecture in 2 layer (how we deploy architecture and how we deploy software)
- Deployment independence: means no dependency on the state, just communication
- Each microservice scale independently. No shared state, in order to be independent we need to have a state, it's the concept of object, the only way 2 objects can interact is through interface (no global variables).
- Safety: there is someone that can look at the state of services
- Scalability
- Layered vs Microservices: Do not confuse MVC-level patterns with pattern-compliant microservices. But we can organize services in layers, but we don't have layers in the sense of object programming. Design patterns are not the pattern of architecture.
- Independence, that is not just independence in term of execution but in term of:
 - Code: Dependencies and development process (repository). No more dependent by library, but we can change without problem
 - Scalability: Specific and independent technology stacks. Reason to scale must be separated.
 - Functionality: Separate evolution and differentiated integration. I can't modify the function until I modified the interface. Changing interface is something that have an impact on other component.
- Standard communication (protocols): for supporting independency, this is why we decided to go for HTTP over RPC

Monolith is connected to other application, and it also can be structured in layers, but it's not deployed as services. Services have their own interface, can exist independent. But the layer of connection is a nightmare, in term of code could be called spaghetti code. Still we have API expose to the rest of the world. What is not evident is the dependencies of infrastructure like what we have on monolithic (MySQL), the idea is trying to split the services, why they need the same instance of MySQL to save data.



Other requirements:

- Decentralization:
 - Governance
 - Data management
- Automation: is a service is not working it has to be ri put up:
 - Continuous delivery: need outomation testing code and put in service
 - Monitoring
 - Management
 - Rapid provisioning
 - Traceability of deployably rollback
- Versioning of APIs: usually we have 2 type of API, the internal and the external (offer our service for the rest of the world, huge business in API development):
 - Compatibility, backward and forward
 - Stable contracts

Specific support for these requirements comes from virtualization:

- Replicability of environments
- Protection by isolation

- Distribution and coexistence: allow us to scale (both horizontal, adding services, then vertical, improving the capability of machine)
- Automation with recipes: we can put automation scripts (for code, fixing issues..)

Granularity manageable through various types of virtualization.

Runtime Options:

- Instead of services on shared OS that generate problems of incompatibility of libraries:
 - Specific containerized environments: bring to microservice what is needed to be executed
 - Multiple microservices on the same host
 - Fault resistance
- Containers encapsulate a runtime environment:
 - Direct to HW
 - On virtual machine

On container we can obtain:

- Isolation
- Best performance
- Sharing at OS level

One important aspect is that not all the components can really stay together without making an issue, for example we can't have "CPU intensive" processes on one machine.

Microservices architecture does not oblige the use of containers. Netflix used AWS without container at the beginning. They are effective from the point of view of reactivity at variable loads. Profits in the startup phase to reduce costs and improve sharing. There are things to consider especially compromise between software and system architecture. In theory we have data and services that can be everywhere, but de facto is useful to DEPLOY the service near the data in order to obtain performance (edge computing). Deploying could be dynamics.

What we have to handle in the start time:

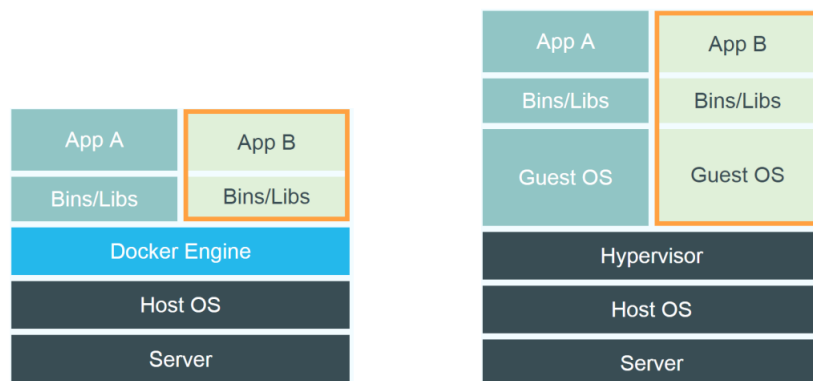
- Tools and new processes to manage:
 - Complex system of containers that scale over time
 - Advanced monitoring to be used
- Manage the orchestration
- Service Discovery (where required)
- Greenfield project: No legacy restrictions from the beginning
- One microservices per container
- Container security: Trusted registries

6.2 Containers Technology

This is not a completely new thing, started 40 years ago. Containers aim to control access to resources such as the file system and arrive at complex control of CPU, memory, network etc. To control access the naming of resources is fundamental, name space.

6.3 Docker

It encloses a piece of software in a complete filesystem containing everything that is needed to run the software itself.



Leverages decades of containment technology development:

- Libcontainer
- Lxc
- Namespace
- Control groups
- Union filesystem (union mount): layered filesystem

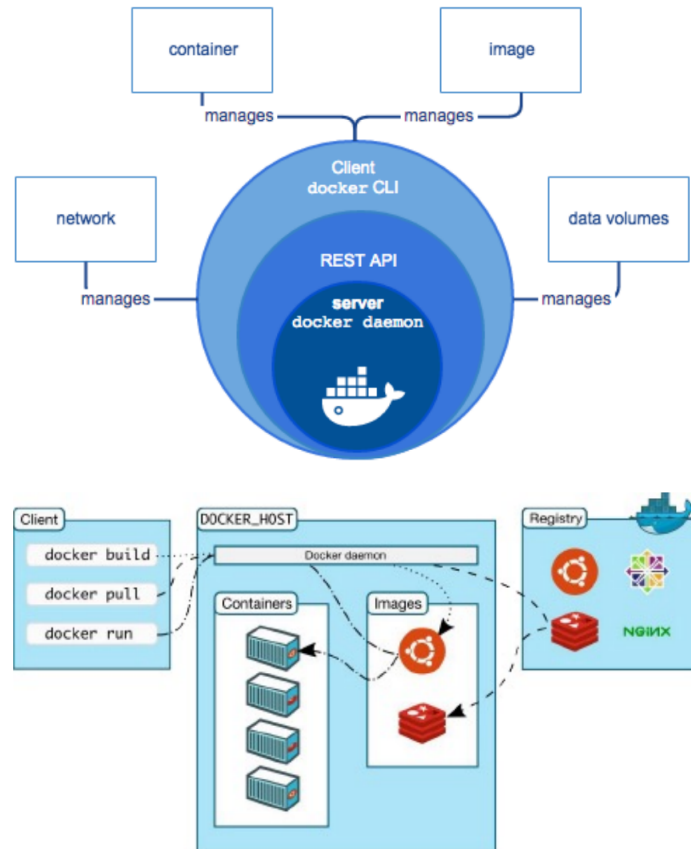
From external entities that connect to the docker we have:

- Server: a daemon (dockerd)
- REST API
- CLI interface (docker)

6.3.1 Architecture

Docker consists of and parts:

- Registry contains images of containers
- Docker daemon query the registry to get the image to run in a container



- Docker client communicates with docker daemon to instructs it on actions to be taken

Fundamentals:

- Host: the machine running the container on which the docker daemon runs
- RegistryHub: image repository:
 - Public or private
 - Official: <https://hub.docker.com/>
- Image: file hierarchy, with meta-data to run the container. It's based on union file system, based on the layer.
 - Use a Union filesystem to aggregate multiple layers to present them as a single filesystem
 - Structured to levels (layer): Do not duplicate filesystem to each instance
 - Readonly, once created a image layer it cannot be modified (but it is possible to overwrite it with another layer): Isolate changes to the container layer
 - Use cache mechanisms for the layer
 - Has a unique ID
 - It has one or more tag to identify the version

Basic commands for Image:

```
#docker images //shows the images in the system
#docker save //save image to tar archive
#docker import //creates the image from a tar archive
#docker build //creates the image from a Dockerfile
#docker commit // creates the image from a container
#docker rmi //remove the image
#docker history //show image history
```

Container is a running image:

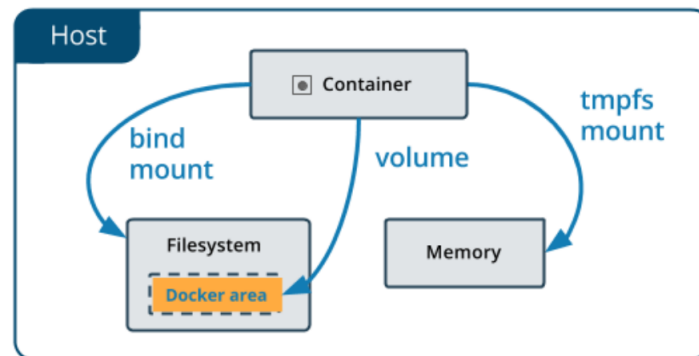
- The running image, contains the application and all the libraries and data needed to run it
- Has its own IP
- Works with the mechanism of copy-on-write: each file is in common with the others
- Exposes:
 - Port to communicate with the outside world
 - Volumes to save your data
 - ENV possible environmental variables needed for the application performed inside the container

Basic command for container:

```
#docker create // creates a container (but doesn't start it)
#docker run //creates and starts a container
#docker stop //stop a container
#docker start //start a container
#docker restart //restart a container
#docker rm //remove a container
#docker kill //send SIGKILL to container
#docker attach //connects to a running container
#docker wait //wait until container ends
#docker exec //runs a command in a container
```

Volume: storage volume shared by the container:

- Easy backup or migrate
- Can be managed with Docker CLI
- Sharing between multiple containers



`#docker volume create my-vol//create volume`

`#docker volume ls //list volumes`

`#docker volume inspect my-vol//inspection a volume`

`#docker volume rm my-vol //removes a volume`

Dockerfile script to create the image:

- Script that allows the creation of an image
- Each statement creates a layer
- Automate operations for a single image

After developing a software must be deployed.

- Infrastructure standardization: Create a standard, scalable infrastructure to run every application.
- Continuous Integration build Isolation: simplifies dependency management.
- Testing: for example verify a language's version change.
- Deploy new software: create and load the entire docker image on the repository.

6.4 Dockerfile

Dockerfile is like a make file, it's listing the activity for building an image. Usually we start from an existing image using FROM command:

- FROM nginx // basic image to start with it is the default command
- COPY hello.txt /usr/share/nginx/html

Let's see an example of dockerfile for building a simple nodejs server:

```
FROM debian:latest
RUN apt-get update -y
RUN apt-get install -y curl
RUN curl -sL https://deb.nodesource.com/setup_12.x | bash
RUN apt-get install -y node.js
COPY server.js /
EXPOSE 8080
CMD ["node", "/server.js"]
```

After that we can build docker and run it:

```
# docker build -t ahawkins/docker-intro-hello
```

```
# docker run -d -p 8081:8080 ahawkins/docker-intro-hello
```

In deploying we are not installing anything in our machine, but we are just listing the command that we want to install on docker and just install docker.

We can generate more complex image:

```
# FROM debian:latest
RUN apt-get update -y
RUN apt-get install -y curl
RUN curl -sL https://deb.nodesource.com/setup_12.x | bash
RUN apt-get install -y nodejs
RUN mkdir -p /app
WORKDIR /app
COPY package.json /app
RUN npm install
COPY ./app/
EXPOSE 8080
CMD ["npm", "start"]
```

The json contains what to do in relation with the start request:

```
"start": "node server.js"
```

There are official images containing nodejs. It is a practical way to simplify the complexity of the dockerfile.

6.5 DOCKER TOOLCHAIN

- We have docker engine that is capable to executing and managing containers (create network, store data..)
- We have docker compose that is a way to activate multiple containers in the same time and orchestrate them. It's made via YAML file.
- Docker machine is no longer available, it's way to create a docker host and handling machines hosting docker engines.
- Docker Swarm: a tool for developing Docker clusters
- Docker registry: Docker images repository
- Docker Data center: combination of the ones above
- Docker Cloud: It's a kind of hosted Docker swarm. Given a machine on the public cloud, it connects it to the Docker Cloud. It then can be used to scale applications

6.5.1 Docker Compose

It's kind of docker service called by docker itself. It's used to run dockers all together. If we have an application that has to make and run different docker with this service we can describe the composition and run in one time. It's possible to monitor.

Three steps:

- Define the Dockerfile so that is playable anywhere
- Define the services that run the application in the docker-compose.yml so they can run together in an isolated environment
- Run docker-compose up which launches the whole application

We have different docker that are isolated (different, system, different versions), imagine to do that in the same machine if we need same system that require different versions of the same library it's impossible, but in case of docker we can make two isolated system. It's largely used in devops and continuous integration of deployment.

Preserve data in volumes when containers are created, When it starts, if it finds containers from the previous run, it copies the volumes into the new containers so as to avoid data loss.

The compose keeps track of the configurations used to create a container. When you relaunch a service that hasn't changed, then Compose reuses the existing containers.

Variable and transportable compositions on multiple environments. Customize compositions according to the environment thanks to variables defined in the composition file.

6.5.2 DevOps

From the developemt point of view we can manage orcestration with yaml file. Containers can share volume, networking, DNS etc. Override the same yaml file to reuse it in different scopes (staging, development, production). The development team can create the orchestration in the yaml file and the other teams can reuse it. Useful to automate testing.

With this compose file we aspect to have two docker:

```
version: '2'
services:
  web:
```

```
    build: .
```

```
  ports:
```

```
    - "5000:5000"
```

```
  redis:
```

```
    Image: "redis:alpine"
```

Then we can build the composition:

```
docker-compose build
```

```
docker-compose up
```

6.6 DOCKER SWARM AND MACHINE

Docker machine was a tool for provisioning and managing dockerized hosts (hosts running the docker engine). It was possible to run the command to point to this new env and then act on the target.

6.6.1 Docker Swarm

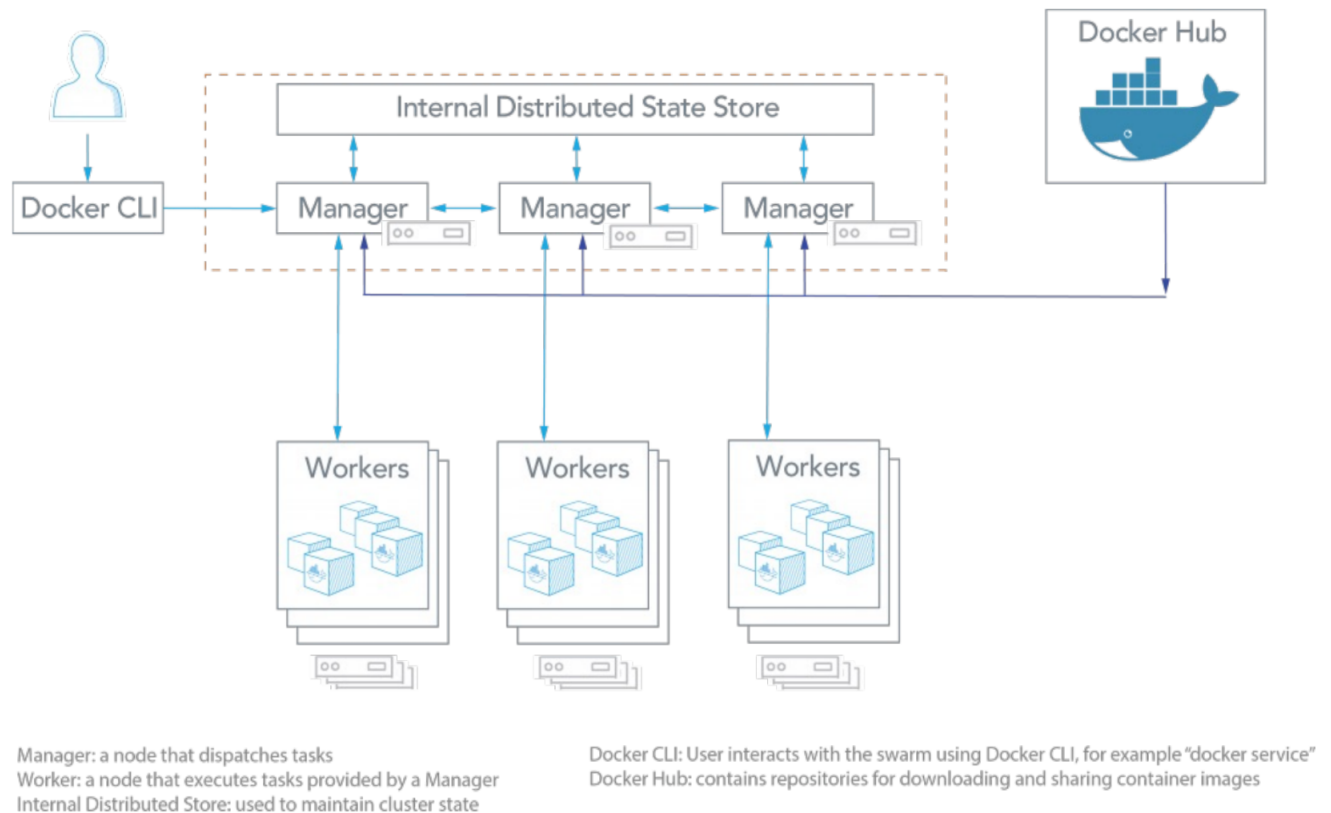
It's possibile to set up docker clusters. Docker Compose allows me to launch docker compositions (microservices) inside these virtual machines. Always confined to a single host whether it be virtual or physical.

Docker Swarm is not different from the view of normal distributed system, a manager has different workers and workers can be everywhere:

Cluster management integrated with Docker Engine: You use the Docker Engine CLI to create a swarm of docker engines to deploy applications. No additional orchestrators.

Scaling: For each service you can declare the tasks you want to run. Swarm handles adding and removing tasks for scale up or scale down. State Reconciliation: Swarm constantly manages nodes to keep requests satisfied. Multi-host networking: It is possible to define an overlay network for services managed by Swarm automatically.

Service discovery: Swarm manages nodes by assigning a single DNS name and load balancer (for expose different instances of the service). It is possible to search containers



via the Swarm DNS. Load balancing: to externally expose and distribute containers across nodes. Secure by default: each node in the swarm requires TLS mutual authentication and encryption to secure communication. Rolling updates: in the production or update phase, it is possible to update the services incrementally.

It allows you to manage clusters and therefore multiple docker hosts Not easy to configure in production It requires a fairly complex yet powerful ecosystem of docker tools.

6.7 KUBERNETES

Nowadays we have more or less a standard for executing multiple images of machines.